Progressive Web Apps

Cours M2 Informatique

Baptiste Colombani

2 octobre 2026

Baptiste Colombani

🔗 linkedin.com/in/baptiste-colombani

📧 baptiste.colombani@gmail.com

Parcours :

2014-2017 – INRIA

2017-2020 – Volpy

2020-2024 – Oscaro Power

Depuis 2024 – Pote&Match



Développeur software & web
Data-scientist
Entrepreneur

Un module, deux parties

Développement d'applications mobiles — M2 Informatique

Partie	Dates	Objectif
I — PWA	ven. 2, ven. 9 et lun. 12 octobre	L'application web installable, hors ligne, notifiée
II — Android	lun. 2, lun. 9 et lun. 30 novembre	La même application, en natif (Kotlin + Jetpack Compose)

Les généralités de ce premier jour valent pour les deux parties :

Généralité	Réponse du web (octobre)	Réponse d'Android (novembre)
Client / serveur, API REST	`fetch()`	Le même serveur, les mêmes requêtes HTTP
Cache et latence	API Cache, Service Worker	Cache HTTP, base locale Room
Fonctionnement hors ligne	Service Worker + IndexedDB	Room + WorkManager
Permissions	Accordées par le navigateur	Accordées par le système

Démo : une PWA sur un téléphone

Support : whatpwacando.today, téléphone projeté

1. **Ouvrir l'URL** dans le navigateur — un site web : une adresse, aucun store, rien à télécharger
2. **Installer** — une icône sur l'écran d'accueil, ouverture en plein écran, sans barre d'adresse
3. **Mode avion, relancer** — l'application s'ouvre et reste utilisable
4. **Déclencher une notification** — elle arrive dans le centre de notifications du téléphone
5. **Caméra, géolocalisation** — le téléphone demande une permission, puis donne accès au matériel

Aucune ligne de code natif : HTML, CSS, JavaScript.

Sur iPhone : installation par Partager → « Sur l'écran d'accueil » (automatique depuis iOS 26) ; les notifications ne fonctionnent qu'une fois l'application installée.

Objectifs

Lundi 12 octobre — fin de la partie PWA

Une PWA par étudiant, sur son propre téléphone :

- **installée** sur l'écran d'accueil
- **utilisable hors ligne**
- **synchronisée** au retour du réseau
- qui **notifie**
- **déployée** en ligne, publiée sur le Class Hub et notée par les pairs

Lundi 30 novembre — fin de la partie Android

La même application, portée en Android natif :

- données locales, synchronisation en arrière-plan
- notification reçue **application fermée**
- **une capacité native au choix** (capteurs, NFC, Bluetooth, widget, caméra…)
- démo croisée : la PWA et l'app Android **côte à côte sur le même téléphone**

Tout ce qui précède le 12 octobre — le Web, l'histoire, les Workers, le cache — sert à construire cette PWA.

Plan du cours (6 jours)

Jour	Matin (cours)	Après-midi (TD/TP)
J1 — ven. 2 oct.	Histoire 2007-2026, test d'une vraie PWA, le Web, les 4 piliers d'une app, SEO/GEO/IA	Qu'est-ce qu'une PWA, Workers, TD
J2 — ven. 9 oct.	Performance, manifest, cache, IndexedDB, offline, notifications	TD offline, notifications
J3 — lun. 12 oct.	TP : la PWA complète	TP (suite), publication, notation croisée
J4 — lun. 2 nov.	Android comme système : sécurité, anatomie d'une app, cycle de vie	TD : la PWA empaquetée, première app Compose
J5 — lun. 9 nov.	Données, arrière-plan, notifications, permissions	TD, lancement du projet

Chapitre 1

Histoire du Web Mobile

2007 : L'iPhone change tout

9 janvier 2007 — Macworld, San Francisco. Steve Jobs présente l'iPhone : un téléphone, un iPod, et « le vrai Safari », pas un navigateur mobile au rabais. Aucun kit de développement n'est annoncé.

11 juin 2007 — WWDC. Aux développeurs qui réclament un SDK, Jobs répond par une « *sweet solution* » :

> "The full Safari engine is inside of iPhone. And so, you can write amazing **Web 2.0 and Ajax apps** that look exactly and behave exactly like apps on the iPhone."

La promesse initiale : les web apps sont l'avenir. Pas de SDK natif prévu — le web suffit.

🎬 **Macworld, 9 janvier 2007 — la présentation de l'iPhone (extrait, 10 min)** · **WWDC, 11 juin 2007 — les web apps (2 min)**

2007 : La vision de Steve Jobs

Ce que Jobs imaginait :

- Apps web en plein écran
- Accès via Safari
- Pas besoin de télécharger
- Mises à jour instantanées
- Un seul code pour tous

Pourquoi c'était logique :

- Apple maîtrise Safari/WebKit *(moteur web créé par Apple, contrôle total)*
- Pas de fragmentation *(1 seul code pour toutes les versions iOS)*
- Sécurité *(le navigateur isole l'app du système)*
- Développeurs web = nombreux *(HTML/JS vs Objective-C)*

Spoiler : ça n'a pas marché comme prévu…

2008 : Le virage App Store

10 juillet 2008 — Lancement de l'App Store

Apple change complètement de stratégie :

- SDK natif (Objective-C)
- Apps téléchargeables
- Modèle économique (30% commission)
- Contrôle total d'Apple

Pourquoi ce revirement ?

- Les web apps de 2007 étaient trop limitées
- Pas d'accès aux capteurs (GPS, accéléromètre)
- Performances insuffisantes
- Pas de mode offline fiable
- Pression des développeurs + communauté jailbreak *(dès juillet 2007, des apps natives "pirates" prouvent la demande)*

2008-2010 : L’explosion des apps natives

Les chiffres parlent :

Année	Apps sur l’App Store
2008	500 apps au lancement
2009	100 000 apps
2010	300 000 apps
2012	650 000 apps

Google suit avec l’Android Market (octobre 2008)

Le web mobile est relégué au second plan.

Le problème des apps natives



Le problème des apps natives

Pour les entreprises :

- Développer iOS = équipe Swift
- Développer Android = équipe Kotlin
- 2 codebases à maintenir
- Coût x2 minimum
- Soumission aux stores (délais, rejets)

Pour les utilisateurs :

- Aller sur le Store
- Chercher l'app
- Télécharger (50-100 Mo)
- Attendre l'installation
- Créer un compte
- Autoriser les permissions

💰 Chaque étape de friction affaiblit le pilier **Activation** *(chapitre 2 : les 4 piliers, tunnel de conversion)*

Le coût du développement natif

Type	Coût estimé	Remarques
Site web responsive	5 000 - 30 000 €	1 seul code, déploiement instantané
PWA	10 000 - 50 000 €	Site web + offline/install
App native (iOS OU Android)	30 000 - 100 000 € chacune	1 équipe par plateforme, soumission store
iOS + Android natif	60 000 - 200 000 €	2 équipes, 2 codebases
Cross-platform (React Native/Flutter)	40 000 - 120 000 €	1 codebase, 2 plateformes

📊 **Le vrai coût caché : la maintenance = 15-25 % du coût initial / an** (une app à 100 k€ ≈ 20 k€/an : MAJ OS, review stores, tests devices). Le cross-platform mature a réduit les coûts initiaux de ~40 % depuis 2018, mais la maintenance reste double. **PWA : déploiement web = MAJ instantanée, pas de review.**

Fourchettes indicatives, app métier moyenne (2024-2026). Très variable selon complexité, agence, localisation.

2010-2014 : Les tentatives hybrides

PhoneGap / Cordova (2009)

- Wrapper natif autour d'une WebView
- Code web → app native
- Promesse : "Write once, run everywhere"

Problèmes :

- Performances médiocres
- Look & feel pas natif
- Bugs spécifiques à chaque plateforme
- Le pire des deux mondes ?

📱 **Et aujourd'hui ?** PhoneGap abandonné par Adobe (2020), Cordova en maintenance. **React Native** (2015) et **Flutter** (2017) les ont remplacés : composants natifs (pas de WebView), meilleures performances. Ils se partagent aujourd'hui le cross-platform (Flutter en tête, React Native second).

2013 : Les OS web tentent leur chance

L'idée radicale : Supprimer la couche "native" et faire tourner TOUT en web (HTML/CSS/JS), même l'OS.
Objectif : casser le duopole iOS/Android et démocratiser les smartphones.

Firefox OS (Mozilla)

- OS entièrement basé sur le web
- Apps = HTML/CSS/JS
- Ciblait les marchés émergents

Ubuntu Touch

- Même philosophie
- Convergence mobile/desktop

Résultat : échec commercial — Pas assez d'apps, performances insuffisantes, écosystème trop faible.

2014 : Le web mobile stagne

État des lieux :

- Sites mobiles = versions dégradées
- "Téléchargez notre app" partout
- Expérience web mobile frustrante
- Les navigateurs mobiles évoluent peu

Les limitations web :

- ❌ Pas d'offline (Service Workers arrivent en 2015)
- ❌ Pas de notifications push (Push API : Chrome 42, avril 2015)
- ❌ Pas d'icône sur l'écran d'accueil
- ❌ Performances médiocres
- ❌ Pas d'accès aux capteurs

2015 : Google relance le web mobile

Juin 2015 — Frances Berriman (designer web britannique) et Alex Russell (ingénieur de l'équipe Chrome, chez Google) introduisent le terme :

"Progressive Web Apps"

> *"Escaping Tabs Without Losing Our Soul"*

L'idée : Les nouvelles APIs navigateur permettent enfin de combler le fossé entre web et natif.

📖 Article original d'Alex Russell (juin 2015)

Pourquoi Google pousse les PWA ?

Intérêt stratégique :

- Google = entreprise du web
- Chrome = navigateur dominant
- Réduire la dépendance aux app stores
- Indexation dans Google Search

Intérêt technique :

- Service Workers (Chrome 40, 2015)
- Push API
- Cache API
- Web App Manifest

Les technologies sont enfin mûres !

2015-2020 : Adoption progressive

Les early adopters (études de cas Google, 2016-2018) :

Entreprise	Résultats
X (ex-Twitter Lite)	-70% data, +65% pages/session
Pinterest	+60% engagement
Starbucks	2x utilisateurs web
Uber	Fonctionne en 2G, **50 Ko seulement**
Alibaba	+76% conversions

Pourquoi ces résultats ?

Point commun : marchés émergents d'abord

Inde, Afrique, Asie du Sud-Est…

- 📶 Connexions lentes (2G/3G)
- 💰 Forfaits data limités et coûteux
- 📱 Smartphones entrée de gamme (peu de stockage)

📊 **Comparaison X (ex-Twitter) :**

- App native (2017) = **~150 Mo** à télécharger
- PWA = **~1 Mo**

→ La PWA est **150x plus légère**

2018 : Apple cède (un peu)

iOS 11.3 (mars 2018)

Apple ajoute enfin le support des Service Workers dans Safari

Mais avec des limitations :

- Pas de push notifications (jusqu'à iOS 16.4 en 2023 !)
- Quota de stockage limité — *~50 Mo pour l'API Cache en 2018, relevé depuis*
- Éviction des données après 7 jours d'inactivité — *Safari efface le cache si l'app n'est pas utilisée*
- Pas de Background Sync — *synchronisation différée quand le réseau revient*

Apple freine volontairement ? 🤔

💰 **L'App Store rapporte à Apple ~30 milliards $ de commissions par an**, sur ~120 milliards $ facturés aux utilisateurs *(estimation Asymco, 2025)*. Les PWA contournent l'App Store → pas de commission. Le conflit d'intérêt est structurel.

La preuve par l'exemple — la saga DMA (2024) :

1. **Février 2024** : Apple annonce la **suppression des web apps d'écran d'accueil dans l'UE**, "pour se conformer au DMA" 🤨
2. Tollé des développeurs ; la Commission européenne répond que c'était "**ni requis, ni justifié**" par le DMA
3. **Mars 2024** : Apple rétropédale… en 2 semaines

Et depuis, le vent tourne :

- Procès **Epic v. Apple** : depuis avril 2025, liens de paiement externes autorisés aux États-Unis, sans commission ; en 2026 la Cour suprême a accepté d'examiner l'appel d'Apple, le taux de commission reste à fixer
- **iOS 26 (2025)** : tout site ajouté à l'écran d'accueil s'ouvre désormais **en web app par défaut**

La pression légale fait plus que la bonne volonté.

2020-2026 : L’ère de la maturité

Aujourd’hui les PWA peuvent :

- ✅ Fonctionner offline
- ✅ Envoyer des notifications (même iOS depuis 2023, simplifié par **Declarative Web Push** en 2025)
- ✅ Accéder au GPS, caméra, micro
- ✅ S’installer sur l’écran d’accueil (**par défaut** sur iOS 26 !)
- ✅ Être publiées sur le Play Store (TWA – *vu au Jour 3 et dans la partie Android*)
- ✅ Utiliser Bluetooth, NFC, USB

Ce qui manque encore (vs natif) :

- ❌ Accès complet au système de fichiers
- ❌ Certains capteurs avancés
- ❌ Performances 3D intensives

Timeline récapitulative

```
2007  iPhone, vision "web apps"
  │
2008  App Store, virage natif
  │
2009  PhoneGap/Cordova
  │
2013  Firefox OS, Ubuntu Touch (échecs)
  │
2015  ⭐ Google introduit "PWA"
  │
2018  Apple ajoute Service Workers (iOS 11.3)
  │
2023  Push notifications iOS (enfin !)
  │
2024  Saga DMA : Apple tente de tuer les web apps en UE, rétropédale
  │
2025  Declarative Web Push · iOS 26 : web app par défaut
  │
2026  Safari 27 : Service Worker static routing · PWA = solution mature
```

La boucle est bouclée

2007 — Steve Jobs :

> "You can write amazing Web 2.0 and Ajax apps"

2026 — La réalité :

> Les PWA réalisent enfin cette vision, 19 ans plus tard

Ce qui a changé :

- Navigateurs beaucoup plus puissants
- APIs standardisées (W3C)
- Performances JavaScript x100
- Connexions plus rapides (4G/5G)



Séance pratique

whatpwacando.today

Le terrain de jeu : whatpwacando.today

Sur téléphone ET desktop → whatpwacando.today

Une PWA qui teste **toutes les capacités** du navigateur : caméra, GPS, notifications, Bluetooth, vibration, partage, badge, stockage, audio…

Mission (20 min), chacun sur son téléphone — les échanges entre voisins sont bienvenus :

1. **Installer** la PWA (sur téléphone ET desktop)
2. Tester au moins **8 APIs** différentes
3. Remplir la grille :

API testée	📱 Téléphone	💻 Desktop	Permission demandée ?
ex : Caméra	✅	✅	Oui
ex : Vibration	✅	❌	Non
…			

4. Objectif : trouver **2 APIs qui marchent sur un device et pas l'autre**
5. **Conserver la grille** : elle sert au Jour 2 et à la partie Android

Débrief

Constat	Ce que ça montre
"Ça marche sur Android, pas sur iPhone"	Les navigateurs n'implémentent pas tous les mêmes APIs → **progressive enhancement**
"On m'a demandé une permission"	Modèle de sécurité du web : **l'utilisateur décide**
"Le badge ne marche que si j'installe"	App **installée** = plus de privilèges
"C'est une app ou un site ?"	…les deux. **C'est exactement ça, une PWA.**

🎯 **Chaque API testée ici est codée pendant le cours** : caméra, GPS, notifs, badge → Jour 2 · offline → Jour 2 · installation → Jour 3.

Côté Android (novembre) : chaque ❌ de la grille est une capacité que le système accorde à une app native — point de départ du Jour 4.

Ce que la grille doit montrer (état 2026)

Partout (Chrome Android et Safari iOS)

Installation · hors ligne · notifications push *(iOS : app installée seulement)* · géolocalisation · caméra, micro · partage · stockage · paiement · orientation, mouvement · synthèse vocale · WebAuthn · View Transitions · picture-in-picture · wake lock · badge *(iOS : installée)*

Chrome Android seulement

Bluetooth · NFC · Background Sync · Background Fetch · vibration · raccourcis d'icône · AR/VR (WebXR) · infos réseau · reconnaissance vocale complète · capture d'écran

Safari seulement

AirPlay · Audio Session · Declarative Web Push

Sur aucun téléphone (Chrome desktop uniquement)

Window Controls · File Handling · Protocol Handling · Link Handling · File System Access complet · Element Capture

Nulle part de façon stable

Détection de visages

Trois navigateurs, trois listes : c'est le sens de **« progressive »** — l'application fonctionne partout, et fait plus là où le navigateur le permet. La colonne orange est le programme du Jour 2 ; la colonne rouge, celui de la partie Android : ce qu'une page ne peut pas faire, le système l'accorde à une app.

Chapitre 2

Le Web

Le Web, en quatre questions



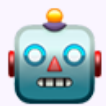
Que se passe-t-il **vraiment** entre une URL tapée et une page affichée ?



Pourquoi le web est-il obsédé par **le cache** ?



Qu'est-ce qui fait qu'une app **marche** ? (ce n'est pas la technique)

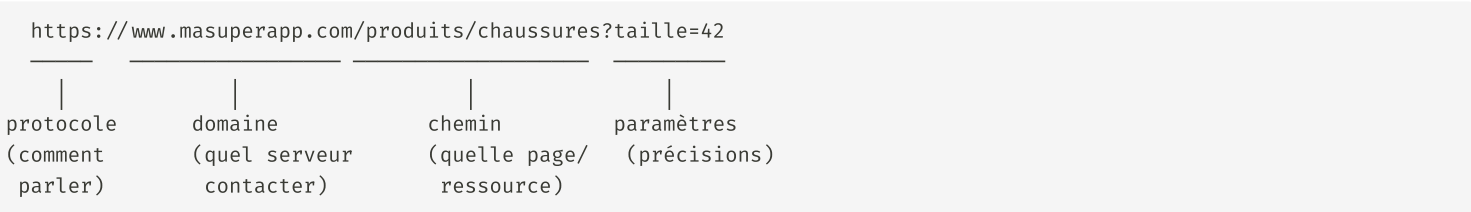


Comment **exister** en 2026, face à Google **et** face aux IA ?

Côté Android (novembre) : ces généralités s'appliquent telles quelles — une app native est un client du même serveur.

Qu’est-ce qu’une URL ?

URL = Uniform Resource Locator = l’adresse d’une ressource sur le web



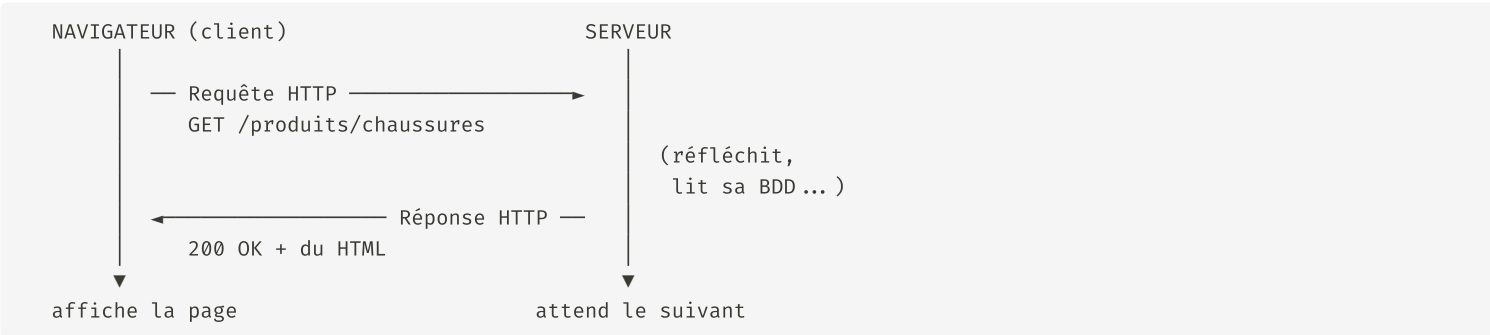
- Le **domaine** est traduit en adresse IP par le **DNS** (l’annuaire d’internet)
- Le **protocole** `https` = HTTP + chiffrement (on y reviendra : obligatoire pour les PWA)

💡 **L’URL est une adresse universelle et partageable** : n’importe quel contenu web s’envoie en un message. Un écran précis d’une app native n’a pas d’équivalent. Propriété centrale du pilier **Acquisition**, détaillé plus loin dans cette introduction.

Client / serveur : la conversation de base

Un serveur = un ordinateur allumé 24h/24, qui attend des requêtes et y répond.

Votre navigateur = le client, qui pose des questions.



Tout le web fonctionne comme ça. Chaque page, chaque image, chaque donnée = une requête → une réponse.

À quoi sert un serveur ?

3 rôles :

Rôle	Exemple
Héberger des fichiers	Le HTML, le CSS, le JS, les images de votre site
Exécuter la logique métier	Vérifier un mot de passe, calculer un panier, enregistrer une commande
Servir tout le monde en même temps	10 000 visiteurs simultanés = 10 000 conversations en parallèle

Deux façons de répondre :

- **Statique** : le fichier existe déjà sur le disque → on l'envoie tel quel *(rapide, simple)*
- **Dynamique** : la réponse est **fabriquée à la volée** — le serveur interroge sa base de données et génère le HTML *(personnalisé, plus coûteux)*

Votre profil Instagram n'existe nulle part en tant que fichier : il est généré à chaque requête.

HTML = une IHM

IHM = Interface Homme-Machine. Et le HTML en est une, à part entière.

App native :

Le code **dessine** ses fenêtres et ses boutons, pixel par pixel, avec le SDK de l'OS

```
Button b = new Button();
b.setPosition(50, 120);
b.draw();
```

Web app :

Le code **décrit** son interface, le navigateur la dessine

```
<button>
  Ajouter au panier
</button>
```

- Le navigateur est un **moteur de rendu d'interfaces** ultra-sophistiqué
- Le **DOM** = votre interface devenue vivante, que le JavaScript peut modifier à tout moment

→ App native et web app : deux moteurs de rendu différents, un même concept d'IHM.

🎬 Démo live 1 : DevTools (F12)

Tout le monde ouvre DevTools – F12 ou clic droit → Inspecter

1. Onglet Elements :

- Le DOM en direct : modifiez le titre d'un site connu, changez une couleur
- *Modification locale uniquement – le serveur n'est pas affecté.*

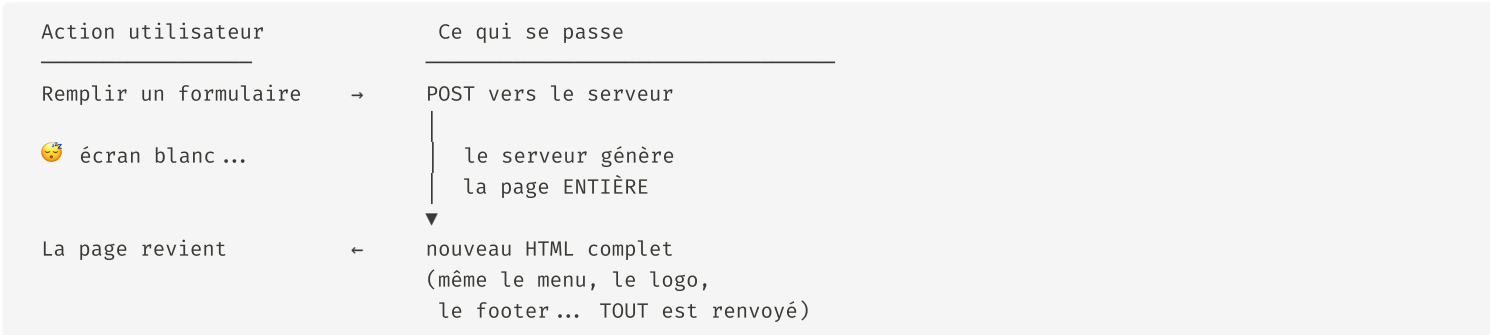
2. Onglet Network – le plus important aujourd'hui :

- Rechargez la page (F5)
- Regardez les fichiers **arriver un par un** : le HTML d'abord, puis CSS, JS, images, fonts…
- Colonnes à observer : **taille**, **temps**, **statut** (200 ? 304 ? 404 ?)

🔬 **L'onglet Network sert pendant les 3 jours du cours** : c'est ici que les fichiers reçus sont visibles – et que le Service Worker les interceptera (J1 après-midi).

1995-2004 : le web page par page

Au début, le web est une machine à documents :



Chaque clic = tout jeter, tout re-télécharger, tout ré-afficher.

Fonctionnel, mais lent, saccadé — rien à voir avec une "application".

2005 : la révolution AJAX

Gmail et Google Maps changent l'histoire du web.

L'idée : ne plus demander des pages, demander des DONNÉES.

```
AVANT (1999)                          APRÈS (2005, AJAX)
─────────────                         ──────────────────
clic → page entière (500 Ko)          clic → juste les données (2 Ko)
       écran blanc                           la page se met à jour
       tout re-télécharger                   sans recharger
```

- Techniquement : `XMLHttpRequest`, aujourd'hui remplacé par `fetch()`
- Format : XML à l'époque → **JSON** aujourd'hui (le X d'AJAX est resté par nostalgie)
- Résultat : la page devient **fluide**, l'expérience se rapproche d'une application

AJAX = le moment où le web a commencé à ressembler à des apps.

🎬 Démo live 2 : AJAX est partout

Ouvrez un site à scroll infini (un réseau social, un site d'actus…) :

1. F12 → **Network** → filtre **Fetch/XHR**
2. Videz la liste (🚫), puis **scrollez**
3. Regardez les requêtes partir **en temps réel** : la page ne recharge jamais, mais elle discute sans arrêt avec le serveur
4. Cliquez sur une requête → onglet **Response** → du **JSON** brut

> 💡 **Ce que vous voyez :** l'interface (HTML/CSS déjà chargés) et les données (JSON à la demande) vivent des vies séparées. **Cette séparation est LA fondation des PWA** – c'est elle qui permettra de mettre l'interface en cache et de fonctionner offline.

La SPA : le HTML devient une coquille

SPA = Single Page Application — la génération d'après (React, Vue, Angular…)

```
<!— Ce que le serveur envoie : —>
<!DOCTYPE html>
<html>
  <body>
    <div id="app"></div>          ← vide !
    <script src="app.js"></script> ← 2 Mo de JavaScript
  </body>
</html>
```

Le JavaScript construit TOUTE l'interface dans le navigateur, puis va chercher les données en AJAX.

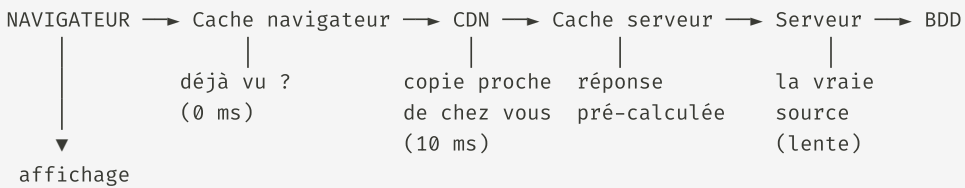
- ✅ Expérience fluide, transitions, état conservé — **comme une app**
- ⚠️ **Premier chargement lourd — et cette coquille vide pose un problème majeur de visibilité (partie SEO/GEO, plus loin dans cette introduction)**

*(Dans la suite du cours, cette coquille aura un nom : l'**App Shell** — et elle sera mise en cache.)*

Le cache : l'obsession du web

Cache = garder une copie d'une réponse déjà obtenue, pour ne pas la redemander.

Pourquoi cette obsession ? Chaque requête coûte : du temps (latence) et de l'argent (serveurs).



À chaque étage, la même question : "est-ce que j'ai déjà la réponse ?"

> *"Il n'y a que deux choses difficiles en informatique : l'invalidation de cache, et nommer les choses."* — Phil Karlton, ingénieur chez Netscape

📅 **Dans la suite du cours (Jour 2) :** l'API Cache — un cache programmable dans le navigateur. L'invalidation de cache fera l'objet du TD « Service Worker zombie » 🧟 .

La latence : l'ennemie invisible

Ordres de grandeur (approximatifs, mais parlants) :

Accès	Temps	À l'échelle humaine*
RAM	~100 ns	1 seconde
Disque SSD	~100 µs	15 minutes
Serveur dans le même pays	~30 ms	3 jours
Serveur aux USA	~120 ms	2 semaines
3G capricieuse en montagne 😅	500 ms - ∞	2 mois - jamais

** si un accès RAM durait 1 seconde*

Conclusion : l'aller-retour réseau est un luxe. Tout le génie du web moderne consiste à l'éviter :

cache (ne pas redemander) · **AJAX** (demander moins) · **offline** (ne rien demander du tout — vu au Jour 2)

Combien de caches entre l'utilisateur et la base de données ?

Étage	Où	Ce qui est gardé	Qui décide	Le piège
Cache HTTP du navigateur	L'appareil	Fichiers et réponses, selon `Cache-Control` , `ETag`	Le serveur (en-têtes)	Un fichier mis à jour, gardé un an
API Cache + Service Worker	L'appareil	Ce que **le code** décide : interface, réponses d'API	Le développeur front – *Jour 2*	L'ancienne version servie sans fin
IndexedDB	L'appareil	Les **données** de l'utilisateur, lisibles hors ligne	Le développeur front – *Jour 2*	Données périmées, conflits au retour du réseau
CDN	Serveurs répartis dans le monde	Fichiers statiques, pages publiques, près de l'utilisateur	L'hébergeur + les en-têtes	Une page **personnalisée** en cache : un utilisateur voit les données d'un autre
Cache serveur (Redis, Memcached)	Le datacenter	Réponses calculées, sessions, requêtes coûteuses	Le développeur back	Modifié en base, encore en cache

Même question à chaque étage : « ai-je déjà la réponse, et est-elle encore bonne ? » Les trois premiers vivent **sur l'appareil** : ce sont eux qui font qu'une PWA s'ouvre en mode avion – une app

Côté serveur : monolithe, microservices ou fonctions ?

	Monolithe	Microservices	Fonctions (serverless)
Principe	Une application qui fait tout : comptes, panier, paiement, catalogue	Des services spécialisés, indépendants, reliés par API	Du code exécuté à la demande par l'hébergeur (Lambda, Cloud Functions, Vercel)
Pour	Simple à développer, déployer, déboguer	Déploiement et montée en charge service par service	Zéro administration, facturé à l'exécution, absorbe les pics seul
Contre	Tout tombe ensemble ; tout redéployer pour une ligne	Réseau, monitoring, cohérence des données	Démarrage à froid, durée limitée, dépendance à l'hébergeur
Pour qui	Un projet qui démarre, 95 % des entreprises	Grande organisation, charges très inégales (Netflix, Uber)	Une API légère, des tâches ponctuelles (notification, image)

⚖️ **Sans dogme :** l'architecture suit la taille de l'équipe et de la charge, pas la mode. Un monolithe bien découpé devient des microservices plus tard ; l'inverse est pénible.

Vu du client, rien ne change : une PWA ou une app Android parle à **une API** par HTTP, quoi qu'il y ait derrière. Le backend du cours (Firebase : Auth, Firestore, FCM) est un serverless

L'API REST : le contrat client ↔ serveur

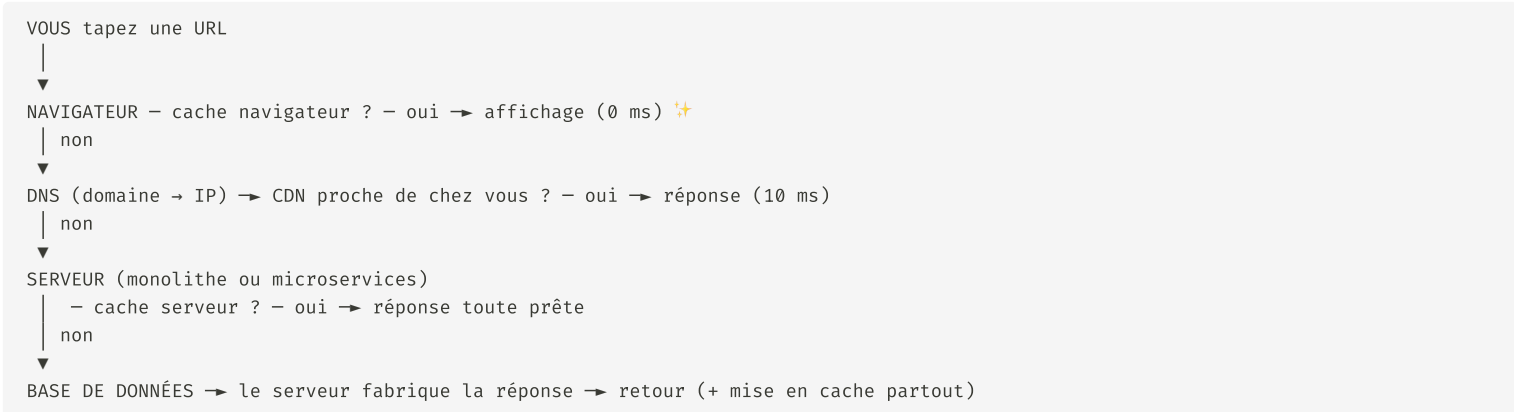
Comment le client et le serveur se comprennent ? Avec des conventions :

Méthode HTTP	Sens	Exemple
`GET`	Lire	`GET /api/articles` → la liste des articles
`POST`	Créer	`POST /api/articles` + JSON → nouvel article
`PUT`	Modifier	`PUT /api/articles/42` → mise à jour
`DELETE`	Supprimer	`DELETE /api/articles/42`

```
// La réponse, en JSON :
{ "id": 42, "titre": "Les PWA", "auteur": "Bat" }
```

Lien PWA : une PWA consomme ce type d'API en continu. Dans la suite du cours : **mise en cache** de ses réponses et **rejeu offline** (Jour 2).

Récap : le voyage d'une requête (30 ans d'optimisations)



Et pendant ce temps, le navigateur, lui, est devenu énorme :

GPS, caméra, micro, notifications, base de données locale, 3D, Bluetooth, paiement…

Le navigateur de 2026 est un OS. Celui de 2007 était une visionneuse de documents.

Une app, du code, et après ?

Ce qui la fait vivre : 4 piliers, des données, des règles

Le cimetière des apps

Vérité qui fâche : une app techniquement parfaite peut mourir en silence.

Les chiffres du cimetière :

- **25 %** des apps installées sont ouvertes **une seule fois**, puis oubliées
- La grande majorité des apps des stores n'atteignent jamais 1 000 téléchargements
- Des millions d'apps "zombies" ne sont plus jamais mises à jour

La bonne question n'est pas :

"Est-ce que mon app tourne ?"

C'est :

*"Est-ce que mon app **vit** ?"* – des gens la trouvent, l'essaient, y reviennent, et elle rapporte de quoi continuer

Ces notions sont rarement enseignées dans les cursus techniques – elles sont pourtant déterminantes.

Les 4 piliers d’une app qui marche

Inspiré du framework AARRR (« pirate metrics ») de Dave McClure — investisseur américain, fondateur du fonds 500 Startups — simplifié en 4 :

Pilier	La question	La métrique clé
1. Acquisition	Comment les gens vous **découvrent** ?	CAC (coût d’acquisition d’un utilisateur)
2. Activation	La **première expérience** est-elle réussie ?	% qui atteignent le "moment magique"
3. Rétention	Est-ce qu’ils **reviennent** ?	Rétention J30 (encore là après 30 jours ?)
4. Revenus	Est-ce que ça **rapporte** de quoi vivre ?	LTV (valeur d’un utilisateur sur sa vie)

🧭 **Fil rouge des 3 jours : chaque technologie PWA du cours sert un de ces piliers.**

Pilier 1 : Acquisition — être découvert

Deux mondes, deux logiques :

📱 Voie Store (app native)

- Chercher parmi **2 millions d'apps**
- ASO*, pub payante, classements
- Télécharger 50-150 Mo avant de voir quoi que ce soit
- **ASO = App Store Optimization, le "SEO des stores"*

🌐 Voie Web

- **Une URL** : partageable en 1 message, cliquable depuis partout
- **SEO** : être trouvé sur Google *(définition plus loin dans cette introduction)*
- **GEO** : être cité par les IA *(définition plus loin dans cette introduction)*
- Zéro téléchargement pour essayer

💰 Acquérir un utilisateur d'app mobile coûte en moyenne **plusieurs euros** (pub) ; un visiteur web organique (SEO) coûte **~0 €** une fois le contenu en place. C'est LE poste de dépense n°1 des startups.

Pilier 2 : Activation — réussir la 1re expérience

Le tunnel de conversion : là où on perd tout le monde

```
100 utilisateurs intéressés
        ▼ Ouvrir le Store          80
        ▼ Trouver l'app            64
        ▼ Cliquer "Installer"      51
        ▼ Attendre le télécht.     41
        ▼ Ouvrir l'app             33
        ▼ Créer un compte          20
```

~80 % de perte avant la première utilisation réelle ! Chaque étape = friction = abandon.

Ennemis de l'activation : téléchargement long, inscription forcée, app lente au 1er écran

Amis de l'activation : essai sans install (URL !), vitesse (Core Web Vitals — vus au Jour 2), valeur immédiate

Illustration conceptuelle — les taux réels varient ([Business of Apps](https://www.businessofapps.com/data/app-conversion-rates/)). L'idée clé : chaque étape coûte des utilisateurs.

Pilier 3 : Rétention — les faire revenir

Le pilier le plus sous-estimé. Sans rétention, l'acquisition est un tonneau percé : vous payez pour remplir, ça fuit en dessous.

Pourquoi les apps natives retiennent mieux qu'un site web classique :

Levier de rétention	Site web classique	App native
Icône sur l'écran d'accueil (rappel visuel quotidien)	❌	✅
Notifications push ("reviens !")	❌	✅
Badge de non-lus sur l'icône	❌	✅
Marche dans le train / en avion	❌	✅

→ C'est exactement cette colonne de droite que la PWA apporte au web.

📅 **Le Jour 2 du cours est entièrement consacré à la rétention** : notifications push, icône, badge, offline — des leviers de rétention avant d'être des features.

Pilier 4 : Revenus — en vivre

Où va l'argent quand un utilisateur paie ?

Canal	Commission (2026)
App Store / Play Store — historique	**15-30 %**
Stores — en pleine évolution*	liens externes autorisés, ~10 % Google, taux Apple en bataille juridique
Web direct (Stripe & co)	**~3 %**

**Procès Epic v. Apple/Google 2025-2026 : les stores s'ouvrent, sous la contrainte des tribunaux et du DMA européen.*

Autres impacts business du web : MAJ instantanée (pas de review de 3 jours) → itérer vite · pas de dépendance à la politique d'un store.

Comment sait-on qu'une app « marche » ?

Par les données qui remontent. Sans mesure, les 4 piliers restent des intentions.

Pilier	Ce qu'on mesure	Comment, concrètement
Acquisition	D'où viennent les visiteurs (Google, lien partagé, IA, pub)	Le `referer`, les paramètres `utm_` dans l'URL
Activation	Combien atteignent le « moment magique »	Un **événement** envoyé par le code : `track("note_créée")`
Rétention	Combien reviennent à J+1, J+7, J+30	Un identifiant stable (cookie ou empreinte) + la date
Revenus	Conversion, panier moyen	Événement `achat` + montant

Deux familles d'outils :

- **Analytics** (fréquentation, parcours, événements) : Google Analytics 4, Matomo, Umami, Plausible
- **Monitoring** (erreurs, lenteurs, crashs) : Sentry, Crashlytics — *partie Android : Android Vitals*

Un script de mesure est un **tiers** chargé sur chaque page : il pèse (Core Web Vitals, Jour 2), il pose souvent un cookie, et il envoie des données quelque part — d'où les trois slides suivantes.

Google Analytics 4 ou Umami ?

	Google Analytics 4	**Umami, Plausible, Matomo**
Éditeur	Google	Open source (Umami : MIT ; Matomo : GPL)
Hébergement	Serveurs Google (États-Unis)	Soi-même (un conteneur Docker, une base Postgres) ou cloud de l'éditeur
Cookies	Oui, identifiant par visiteur	**Aucun** (Umami, Plausible) ; optionnels (Matomo)
Bandeau de consentement (France)	**Obligatoire**	Non requis si l'outil est configuré selon l'exemption CNIL *(Matomo ; vérifier pour Umami/Plausible)*
Poids du script	~90 Ko (gtag.js)	~2 Ko (Umami), ~1 Ko (Plausible)
Ce qu'on obtient	Tout : audiences, publicité, prédictions, liaison avec Google Ads	Pages vues, référents, pays, appareils, événements — l'essentiel
Prix	Gratuit (les données sont la contrepartie)	Gratuit auto-hébergé ; 9-20 €/mois en cloud
Qui voit les données	Google, et vous	Vous

À quoi servent les cookies ?

Un cookie = une petite valeur que le serveur dépose dans le navigateur, et que le navigateur renvoie à chaque requête vers ce domaine. HTTP n'a pas de mémoire ; le cookie en est une.

Usage	Exemple	Posé par
Session : rester connecté	`session_id=a8f3…` après le login	Votre serveur (**first-party**)
Préférences	Langue, thème sombre, panier anonyme	Votre serveur ou votre JS
Mesure	Identifiant de visiteur pour l'analytics	Le script de mesure
Publicité, pistage	Un identifiant lu sur des milliers de sites	Un domaine tiers (**third-party**)

Ce que le développeur contrôle : `HttpOnly` (invisible au JavaScript : vol de session impossible par XSS) · `Secure` (HTTPS seulement) · `SameSite` (pas envoyé depuis un autre site : protection CSRF) · `Max-Age` (durée).

Les cookies tiers meurent lentement : bloqués par Safari (2020) et Firefox ; Chrome a annoncé leur fin pendant quatre ans puis y a renoncé (2024-2025). La publicité se rabat sur le first-party et les empreintes d'appareil.

RGPD : ce que le développeur doit faire

Règlement général sur la protection des données (UE, 2018), appliqué en France par la CNIL. Il porte sur toute donnée qui identifie une personne : e-mail, adresse IP, identifiant de cookie, position GPS.

Les principes qui touchent le code

- **Finalité** : chaque donnée a une raison d'être collectée, écrite quelque part
- **Minimisation** : pas de champ « au cas où »
- **Consentement** pour tout ce qui n'est pas nécessaire au service : cookies de mesure et de pub, newsletter, géolocalisation
- **Durée** : les données ont une date de fin (comptes inactifs, logs)
- **Droits** : accès, rectification, effacement — il faut une route pour les exercer
- **Sécurité** : chiffrement, HTTPS, mots de passe hachés

Concrètement, sur une PWA

- Le bandeau cookies : **refuser aussi facile qu'accepter**, pas de case pré-cochée, choix mémorisé (6 mois), rien d'inessentiel avant le clic
- Notifications push, caméra, position : la permission du navigateur **ne remplace pas** l'information de l'utilisateur sur l'usage
- Un sous-traitant (hébergeur, analytics, Firebase) = un contrat et une localisation des données
- Une page « Politique de confidentialité » qui dit vrai

Ce n'est pas théorique : amendes CNIL de 325 M€ (Google) et 150 M€ (Shein) en septembre 2025, pour des cookies déposés sans consentement. **Côté Android (novembre) :** le formulaire « Sécurité des données » du Play Store rend ces déclarations obligatoires pour publier.

Pilier par pilier : quelle technologie, quel jour du cours

Chaque technologie PWA du cours sert un pilier :

Pilier	Technos PWA	Quand
Acquisition	URL, SEO, GEO	Jour 1 (ce chapitre)
Activation	Vitesse, Core Web Vitals, installation en 1 clic	Jour 2 matin, Jour 3
Rétention	Notifications, badge, offline, icône	Jour 2
Revenus	Pas de store, MAJ instantanées	Jour 3 (déploiement)

Les slides qui suivent — données, cookies, RGPD — valent pour les quatre piliers : mesurer sans données, c'est piloter à l'aveugle ; collecter sans règles, c'est s'exposer à une amende.

Être trouvé en 2026

SEO, GEO, IA

Comment fonctionne un moteur de recherche ?

Google, en 3 étapes :

```
1. CRAWL      Des robots ("crawlers", ex: Googlebot) parcourent le web
   🕷️         en suivant les liens, page par page, jour et nuit

2. INDEX      Chaque page lue est analysée et rangée dans un index géant
   🗃️         (une bibliothèque de plusieurs centaines de milliards de pages)

3. RANKING    À chaque recherche, un algorithme classe les pages :
   🏆         pertinence, autorité, qualité, VITESSE, mobile…
```

Points clés pour la suite :

- Un crawler est un programme **simple et pressé** : il télécharge du HTML, il lit, il repart
- Si votre contenu n'est pas **visible dans le HTML**, il n'existe pas pour lui
- **Les moteurs IA ont aussi leurs crawlers — encore plus limités** *(détaillé dans les slides suivantes)*

Qu’est-ce que le SEO ?

SEO = Search Engine Optimization = tout ce qu’on fait pour apparaître EN HAUT des résultats Google, sans payer.

Pourquoi c’est de l’or :

- Les 3 premiers résultats captent **la majorité des clics**
- Page 2 de Google = le désert
- Trafic **gratuit et durable** (vs pub qui s’arrête quand on arrête de payer)

Les 2 familles :

- **SEO** (organique) : contenu de qualité, technique propre, liens entrants → gratuit mais lent
- **SEA** (payant, ex: Google Ads) : immédiat, mais coûte à chaque clic

💼 Lien avec le pilier **Acquisition** : pour une petite équipe sans budget pub, le SEO est souvent **le seul canal d’acquisition viable**. C’est un avantage structurel du web sur les stores.

SEO d’une PWA : ce qui compte (et pas)

✅ **Ce qui compte :**

- Le contenu **présent dans le HTML** reçu
- La **vitesse** : Core Web Vitals = facteur de ranking officiel *(détaillés au Jour 2)*
- **HTTPS** (déjà obligatoire pour une PWA)
- **Mobile-first** (Google indexe la version mobile)
- `<title>`, meta description, données structurées (schema.org)
- De vraies **URLs propres** par contenu

➖ **Ce qui ne change RIEN au ranking :**

- Le manifest
- Le Service Worker
- Le fait d’être "une PWA"

*Une PWA est un site web : elle est indexée comme un site web. C’est déjà un énorme avantage sur une app native — qui, elle, est **invisible** pour Google.*

⚠️ Le piège SPA : la coquille vide

Rappel de la slide SPA : le serveur envoie `<div id="app"></div>` + du JS…

🎬 **Démo :** clic droit → "Afficher le code source" sur une SPA → chercher le contenu affiché à l'écran : il n'y est **pas**. Puis comparez avec l'onglet Elements (le DOM après exécution du JS) : tout y est.

Qui lit votre page	Exécute le JS ?	Voit votre contenu SPA ?
Un humain avec un navigateur	✅	✅
Googlebot	✅ (avec délai et budget limité)	⚠️ En général oui, mais + lent, + risqué
Tous les autres robots	❌	❌ **RIEN**

Les solutions (mots à connaître) :

- **SSR** – Server-Side Rendering : le serveur envoie le HTML déjà rempli (Next.js, Nuxt)
- **SSG** – Static Site Generation : pages HTML pré-générées au build (Astro…)
- **Prerendering** : générer des snapshots HTML pour les robots

2026 : la recherche devient IA

Une part croissante des questions ne finit plus sur 10 liens bleus, mais sur UNE réponse générée :

AI Overviews (Google) · ChatGPT Search · Perplexity · Claude · Copilot…

📊 **Le chiffre qui change tout :** le recouvrement entre le top 10 Google et les sources **citées par les IA** est passé de ~70 % à **moins de 20 %**.

Traduction : être 1er sur Google ne garantit plus d'exister dans les réponses des IA.

La question de 2026 n'est plus seulement *"est-ce que Google me classe bien ?"***mais aussi** *"est-ce que ChatGPT me cite quand on lui pose une question de mon domaine ?"*

Le GEO : être cité par les IA

GEO = Generative Engine Optimization = tout ce qu'on fait pour être LA source que les IA citent dans leurs réponses.

Le fait technique fondamental :

🤖 **Aucun crawler d'IA n'exécute le JavaScript.** GPTBot, ClaudeBot, PerplexityBot… téléchargent le HTML brut et repartent *(analyse de 500M de requêtes GPTBot : zéro exécution de JS)*. Seul Googlebot rend le JS.

→ Une SPA client-side est totalement INVISIBLE pour ChatGPT, Claude et Perplexity.

Les leviers GEO :

- Contenu dans le **HTML initial** (SSR/SSG — encore eux !)
- Contenu **citable** : chiffres, sources, structure claire type question/réponse
- **schema.org** (données structurées) pour être compris des machines
- Vérifier **robots.txt** et Cloudflare : beaucoup de sites **bloquent les bots IA sans le savoir**
- `llms.txt` : standard émergent pour se présenter aux IA

Apps natives vs PWA, face à l'IA

Réfléchissez : quand une IA répond à une question, elle cite… des pages web.

📱 App native

- Contenu enfermé dans l'app : **pas d'URL**
- Invisible pour Google, invisible pour les IA
- Un agent IA ne peut pas "naviguer" dans votre app
- Votre savoir-faire n'alimente aucune réponse

🌐 PWA / site web

- Chaque contenu a une **URL**
- Crawlable, indexable, **citable** dans une réponse IA
- Un agent IA peut la visiter, la lire, la recommander
- Chaque citation = de l'**Acquisition gratuite**

🚀 **L'argument de 2026 :** les IA et leurs agents naviguent le **web**, pas les stores. Miser sur le web, c'est être visible là où les réponses se fabriquent désormais.

Checklist SEO/GEO — à ressortir pour vos TP

Checklist applicable aux PWA du TP (lundi) :

- ☐ Le contenu important est **dans le HTML** que le serveur envoie (testez : code source, pas Elements !)
- ☐ Un `<title>` et une meta description **par page**
- ☐ Des **URLs propres** : `/recettes/tarte-citron` plutôt que `/#/page?id=42`
- ☐ **schema.org** basique (Article, Product, FAQ…)
- ☐ `robots.txt` : décidez **consciemment** qui peut vous crawler (Google ? GPTBot ?)
- ☐ Vitesse : Core Web Vitals au vert *(vus au Jour 2)*
- ☐ HTTPS ✅ (obligatoire pour la PWA de toute façon)

Résumons

Le navigateur est devenu un OS.

Le web gagne sur l'acquisition, les marges, et la visibilité IA.

Il lui manque ce que les apps ont : l'icône, le hors ligne, les notifications.

Chapitre 3 : ce que la PWA ajoute au web.

Chapitre 3

Qu’est-ce qu’une PWA ?

PWA = Progressive Web App

Une définition simple :

> Une PWA est un **site web** qui utilise les **technologies modernes du navigateur** pour offrir une **expérience similaire à une app native**.

Site web

HTML, CSS, JavaScript

Accessible via URL

+

APIs navigateur

Service Workers, Cache, Push…

=

Expérience native

Offline, installation, notifications

Ce qu’une PWA EST – et N’EST PAS

✅ **Une PWA EST :**

- Un **site web évolué** – 100 % HTML/CSS/JS
- Des **APIs navigateur standards** (W3C, WHATWG, WICG)
- Une **philosophie** : "Progressive Enhancement", s’adapter aux capacités
- **Accessible via une URL** – partageable, indexable
- **Installable** (optionnellement), sur l’écran d’accueil
- **Fonctionnelle offline** – Service Worker + cache

❌ **Une PWA N’EST PAS :**

- Une technologie unique – c’est un **ensemble** combiné
- Un framework – React, Vue, Angular… **optionnels**
- Un standard officiel W3C – terme **marketing** (Google, 2015)
- Une certification – pas de label "PWA officielle"
- Obligatoirement installée – marche aussi dans l’onglet
- Une app native déguisée

Les 3 caractéristiques clés

🚀 Capable

Accède aux fonctionnalités du device :

- Caméra, micro
- GPS, accéléromètre
- Bluetooth, NFC
- Notifications push

💪 Fiable

Fonctionne dans toutes les conditions :

- Pas de réseau → offline
- Réseau lent → cache
- Réseau instable → résilience

📱 Installable

S'intègre au système :

- Icône écran d'accueil
- Plein écran (pas de barre URL)
- Splash screen au lancement

Pourquoi "Progressive" ?

Le principe d'amélioration progressive :

L'app s'adapte aux capacités du navigateur. Pas de "tout ou rien".

```
┌──────────────────────────────────────────────────────────┐
│ Chrome / Edge / Firefox récent                           │
│ → Toutes les fonctionnalités : offline, notifications, install │
├──────────────────────────────────────────────────────────┤
│ Safari iOS                                               │
│ → Presque tout, sauf quelques APIs (ex: Background Sync) │
├──────────────────────────────────────────────────────────┤
│ Navigateur ancien                                        │
│ → Site web classique, toujours fonctionnel               │
└──────────────────────────────────────────────────────────┘
```

Le site ne "casse" jamais – il offre juste moins de fonctionnalités sur les vieux navigateurs.

Les briques technologiques d'une PWA

Technologie	Rôle	Standard
Service Worker	Script qui tourne en arrière-plan, intercepte les requêtes réseau	W3C
Web App Manifest	Fichier JSON qui décrit l'app (nom, icônes, couleurs)	W3C
Cache API	Stocke les fichiers (HTML, CSS, JS, images) pour l'offline	W3C
IndexedDB	Base de données locale dans le navigateur	W3C
Push API	Reçoit des messages push depuis un serveur	W3C
Notifications API	Affiche des notifications système	WHATWG
Background Sync	Synchronise des données quand le réseau revient	WICG

Toutes ces APIs sont natives au navigateur — aucun plugin nécessaire.

PWA vs Site Web vs App Native

Critère	Site Web	PWA	App Native
Installation	❌ Non	✅ 1 clic	⚠️ Via Store
Offline	❌ Non	✅ Oui	✅ Oui
Notifications push	❌ Non	✅ Oui	✅ Oui
Accès capteurs	⚠️ Limité	✅ Étendu	✅ Complet
Mise à jour	✅ Instantanée	✅ Instantanée	⚠️ Via Store
Coût de dev	€	€	€€€
Référencement SEO	✅ Oui	✅ Oui	❌ Non
Distribution	URL	URL	Store uniquement

Critères techniques d’installation

Pour qu’une PWA soit "installable", il faut :

HTTPS

Sécurisé obligatoirement

(localhost OK pour dev)

Manifest valide

Fichier `manifest.json`

avec nom + icônes

Service Worker

Enregistré et actif

avec un `fetch` handler

Note : Ces critères sont le **minimum**. Une bonne PWA doit aussi fonctionner offline, charger vite, et être responsive.

🔬 DevTools → Application : le QG de votre PWA

Chrome → F12 → onglet "Application" – l'outil qu'on utilisera pendant 3 jours

Section	Ce qu'on y voit	Ce qu'on y fait
Manifest	Nom, icônes, couleurs, erreurs/warnings	Vérifier l'installabilité
Service Workers	Statut : activé, en attente, erreur	Update, Unregister, simuler **Offline**
Cache Storage	Contenu de chaque cache (URL → réponse)	Inspecter, supprimer des entrées
IndexedDB	Databases, stores, données	Lire, modifier, supprimer

💡 Les logs d'un Service Worker n'apparaissent **pas** dans la console de la page : **Console → menu déroulant "top" → choisir sw.js**, ou Application → Service Workers → "Inspect".

🎬 **Démo sur m.uber.com :** Application → Manifest ✓ → Service Workers ✓ → cocher "Offline" → recharger → ça marche encore !

Démo : Installer une PWA

Sur Chrome (Desktop) :

1. Aller sur m.uber.com
2. Icône "Install" dans la barre d'URL (ou ⋮ → Installer)
3. Cliquer "Installer"
4. L'app s'ouvre dans sa propre fenêtre

Sur Android (Chrome) :

1. Aller sur m.uber.com
2. Bannière "Ajouter à l'écran d'accueil" OU Menu ⋮ → Installer
3. Icône apparaît sur le home screen

Sur iOS (Safari) :

1. Aller sur m.uber.com
2. Bouton partage (carré + flèche) → "Sur l'écran d'accueil"
3. Confirmer → Icône sur le home screen

Chapitre 4

Les Workers

Le problème : JavaScript est mono-thread

Un seul fil d'exécution pour tout : le JavaScript, le DOM, les événements, les animations. Quand ce fil est occupé, **rien d'autre ne se passe**.



17.0 s

Compteur : 0

Calcul lourd sur le fil principal (3 s)

Le carré et l'horloge sont animés par JavaScript. Lancer le calcul et les regarder.

```
// Ce code bloque TOUTE la page pendant 3 secondes
const end = performance.now() + 3000;
while (performance.now() < end) { /* calcul */ }
```

Le carré s'arrête, l'horloge se fige, le compteur ne répond plus : le navigateur n'a qu'un fil pour exécuter ce code **et** dessiner. *(Le carré est tourné par `requestAnimationFrame` : une animation CSS `transform`, elle, continuerait – le navigateur la joue sur un autre fil, le compositeur.)*

Démo : le blocage du thread principal

Ouvrir la démo : `demos/demo-thread-bloquant/index.html`

La page contient :

1. Un **compteur** avec bouton "Incrémenter"
2. Un **carré qui tourne** (animation CSS)
3. Un **bouton rouge** qui lance un calcul de 5 milliards d'itérations

L'expérience :

1. Cliquez sur "Incrémenter" → ça marche ✅
2. Regardez l'animation → elle tourne ✅
3. Cliquez sur le bouton rouge 🔴
4. **Pendant le calcul (~5-10s)** : impossible de cliquer, l'animation freeze, la page ne répond plus ❌
5. Quand c'est fini : tout redevient normal

→ C'est le problème du mono-thread !

Mais… et Ajax ? Et les Promises ?

Question légitime : "Pourquoi un Worker ? Ajax et `Promise.all` ne suffisent pas ?"

Non ! Car ils ne résolvent pas le même problème :

Solution	Résout quoi ?	Exemple
Ajax / fetch	Attente réseau (I/O)	Requêtes API
Promise.all	Attendre plusieurs choses en parallèle	Plusieurs appels API
setTimeout	Découper en petits morceaux	Animations
Web Worker	**Vrai calcul parallèle**	Traitement lourd

La différence clé : ATTENDRE vs CALCULER

`Promise.all` **= on ATTEND des choses externes**

```
// Le serveur travaille, pas nous → thread libre → UI ok
const [users, products] = await Promise.all([
  fetch('/api/users'),     // Attend le serveur
  fetch('/api/products')   // Attend le serveur
]);
```

C'est comme commander 3 pizzas : tu attends, mais tu peux faire autre chose.

Calcul lourd = on TRAVAILLE nous-mêmes

```
// NOUS on travaille → thread occupé → UI bloquée
for (let i = 0; i < 5_000_000_000; i++) {
  result += Math.sqrt(i);  // Notre CPU calcule !
}
```

C'est comme faire 3 pizzas toi-même : impossible de faire autre chose.

Visualisation : attendre ou calculer ?

ATTENDRE
fetch, Promise.all

UI

CALCULER
boucle, parsing, tri

UI

Lancer

`await fetch()` rend la main au navigateur pendant que le serveur travaille : le fil principal est libre. Une boucle de calcul ne rend jamais la main : il faut un **Worker** pour calculer ailleurs.

Quand utiliser quoi ?

Situation	Solution	Pourquoi
Appels API	`fetch` + `Promise.all`	On **attend** le serveur
Lecture fichier distant	`fetch`	On **attend** le réseau
Timer / délai	`setTimeout`	On **attend** le temps
Parsing d'un gros JSON (Mo)	**Worker**	On **calcule**
Traitement d'image	**Worker**	On **calcule**
Crypto / hash	**Worker**	On **calcule**
Tri de millions d'éléments	**Worker**	On **calcule**

La règle : Si c'est **notre code** qui fait le travail lourd → **Worker**

La solution : les Web Workers

Exécuter du JavaScript sur un fil séparé. Le fil principal garde le DOM, l'interface et les événements ; le worker reçoit le travail lourd et renvoie le résultat par `postMessage()`.



17.0 s

Compteur : 0

Calcul lourd sur le fil principal (3 s)

Le même calcul dans un Web Worker

Le carré et l'horloge sont animés par JavaScript. Lancer le calcul et les regarder.

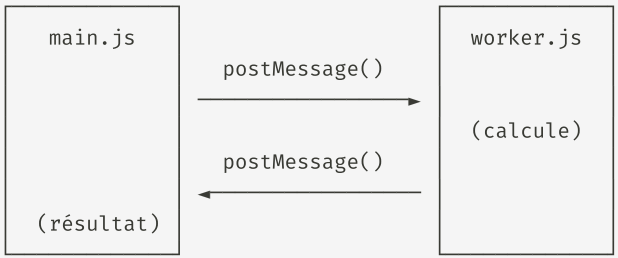
```
MAIN THREAD                              WORKER THREAD
DOM · UI · événements · animations  ◄──► calculs · parsing · traitement de données
                    postMessage() / onmessage
```

Les 3 types de Workers

Type	Portée	Durée de vie	Cas d'usage principal
Web Worker	1 page	Tant que la page est ouverte	Calculs lourds, parsing
Shared Worker	Plusieurs onglets	Tant qu'un onglet l'utilise	Sync entre onglets
Service Worker	Tout le site (scope)	Persiste même fermé	Offline, cache, push

Web Worker : principe

Communication par messages :



Pas de mémoire partagée — messages copiés

Web Worker : structure des fichiers

```
/mon-projet
│
├── index.html
├── main.js        ← Code de la page
└── worker.js      ← Code du worker (thread séparé)
```

Règle importante :

Le fichier worker doit être accessible via HTTP (pas de file://)

→ Utiliser un serveur local pour tester

Web Worker : index.html

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Web Worker Demo</title>
</head>
<body>
  <h1>Web Worker</h1>

  <input type="number" id="nombre" value="45">
  <button id="btn">Calculer Fibonacci</button>

  <p>Résultat : <span id="result">—</span></p>
  <p>Essayez de cliquer ailleurs pendant le calcul !</p>

  <script src="main.js"></script>
</body>
</html>
```

Web Worker : main.js

```
// Créer le worker
const worker = new Worker('worker.js');

// Références DOM
const btn = document.getElementById('btn');
const input = document.getElementById('nombre');
const result = document.getElementById('result');

// Envoyer une tâche au worker
btn.addEventListener('click', () => {
  const n = parseInt(input.value);
  result.textContent = 'Calcul en cours...';
  worker.postMessage({ nombre: n });
});

// Recevoir le résultat du worker
worker.onmessage = (event) => {
  result.textContent = event.data;
};

// Gérer les erreurs
worker.onerror = (error) => {
  console.error('Erreur worker:', error.message);
};
```

Web Worker : worker.js

```
// Le worker écoute les messages de la page principale
self.onmessage = (event) => {
  const n = event.data.nombre;

  console.log('Worker: calcul de fibonacci(' + n + ')...');

  // Calcul lourd (récursif non optimisé volontairement)
  const resultat = fibonacci(n);

  // Renvoyer le résultat à la page principale
  self.postMessage(resultat);
};

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Web Worker : ce qu'il peut / ne peut pas

✅ **Accessible dans un Worker :**

- `self` (référence globale)
- `postMessage()` / `onmessage`
- `fetch()` / `XMLHttpRequest`
- `setTimeout()` / `setInterval()`
- `IndexedDB`
- `WebSockets`
- `importScripts()`

❌ **NON accessible :**

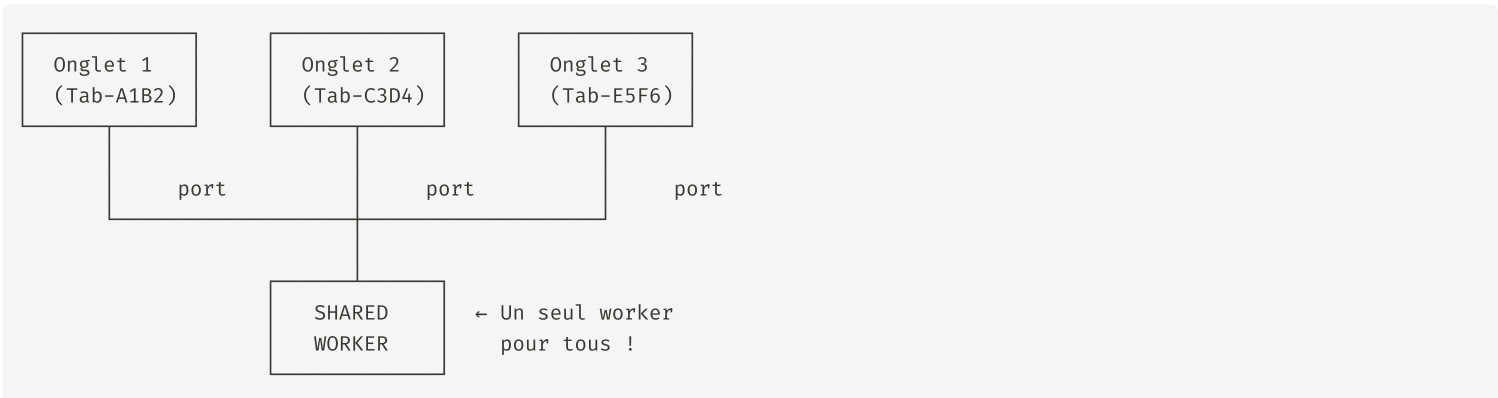
- `document` (pas de DOM)
- `window`
- `parent`
- `localStorage`
- `alert()` / `confirm()`
- Tout ce qui touche à l'UI

📖 **Documentation :** MDN – Web Workers API | postMessage() | Transferable Objects

Shared Worker : partage entre onglets

Le problème : Comment synchroniser des données entre plusieurs onglets du même site ?

La solution : Un Shared Worker = un seul worker partagé par tous les onglets



Démo : `demos/demo-shared-worker/` – Ouvrir dans 2-3 onglets et envoyer des messages

Shared Worker vs Web Worker

Critère	Web Worker	Shared Worker
Portée	1 page = 1 worker	Tous les onglets partagent 1 worker
Communication	`worker.postMessage()`	`worker.port.postMessage()`
Événement connexion	–	`self.onconnect`
Durée de vie	Meurt avec la page	Meurt quand tous les onglets ferment

Cas d'usage :

- Chat synchronisé entre onglets
- Panier e-commerce partagé
- Connexion WebSocket unique (économie de ressources)

Shared Worker : côté page (main.js)

```
// 1. Créer une connexion au Shared Worker
const worker = new SharedWorker('shared-worker.js');

// 2. Démarrer le port (obligatoire !)
worker.port.start();

// 3. Envoyer un message au worker
function sendMessage(text) {
  worker.port.postMessage({
    type: 'broadcast',
    message: text
  });
}

// 4. Recevoir les messages du worker
worker.port.onmessage = (event) => {
  const data = event.data;
  console.log('Message reçu:', data.message);
  // → Afficher dans l'interface
};
```

Différence avec Web Worker : on passe par `worker.port` (pas directement `worker`)

Shared Worker : côté worker (shared-worker.js)

```
const connections = [];  // Tous les onglets connectés

self.onconnect = (event) => {
  const port = event.ports[0];
  connections.push(port);

  port.onmessage = (e) => {
    // Broadcast à TOUS les onglets
    connections.forEach(p => p.postMessage(e.data));
  };

  port.start();
};
```

Points clés :

- `self.onconnect` → appelé pour chaque nouvel onglet
- `connections` → liste de tous les ports (onglets)
- `forEach` → envoie le message à tout le monde

Service Worker : le plus puissant

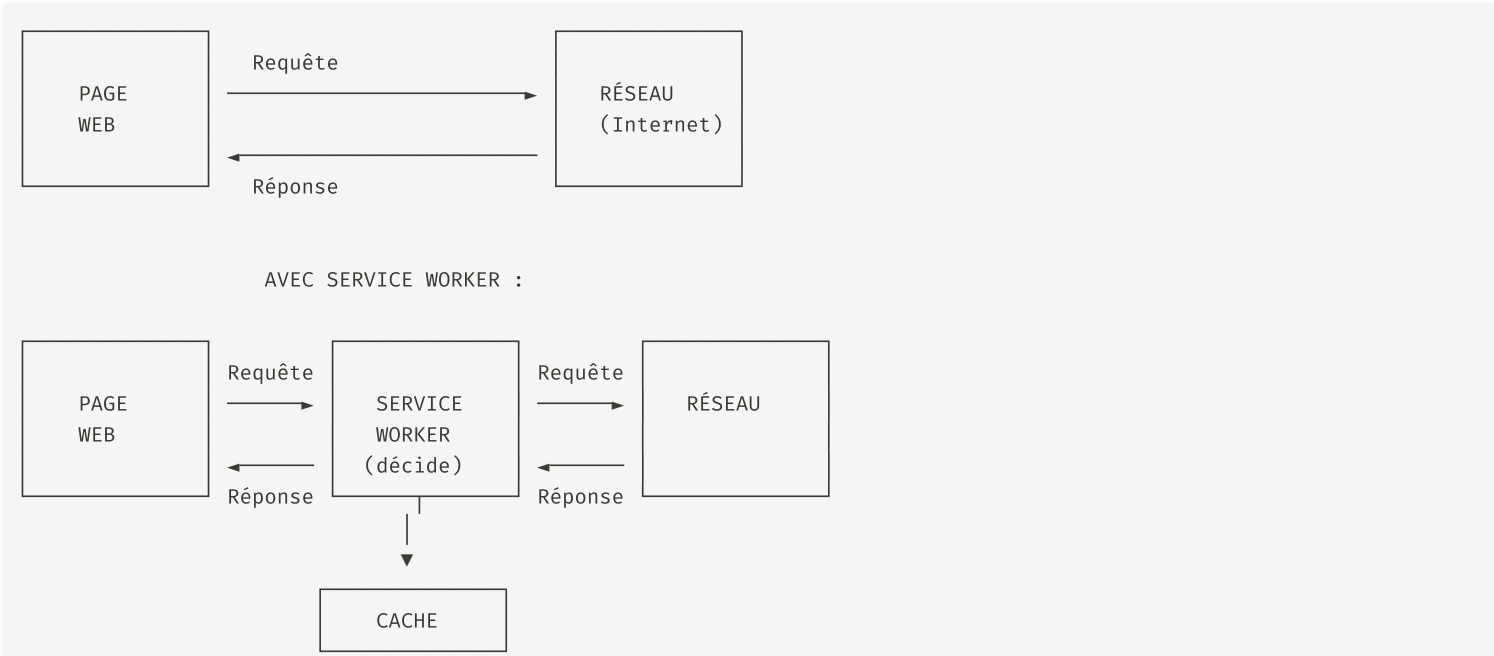
Définition W3C :

> "A service worker is a type of web worker that acts as a **programmable network proxy**, allowing developers to control how network requests from the page are handled."

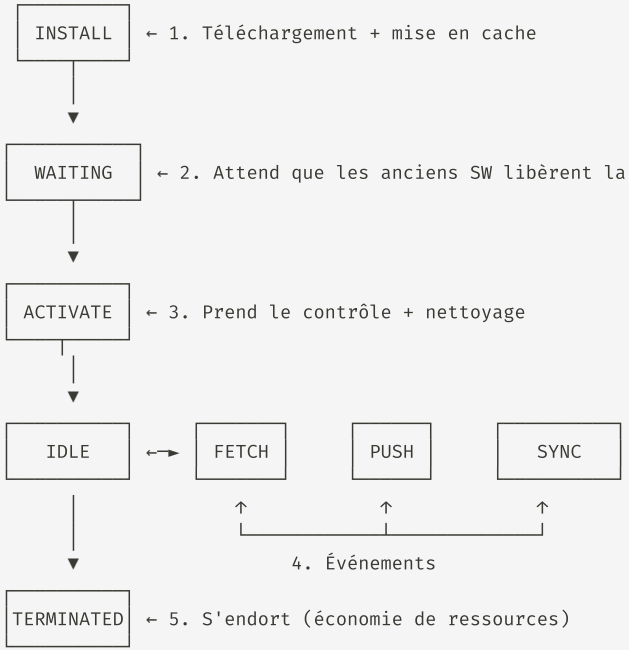
En clair :

Un script qui s'intercale entre votre app et le réseau, et qui peut décider quoi faire de chaque requête.

Service Worker : le proxy réseau



Service Worker : cycle de vie (vue d'ensemble)



Cycle de vie : 1. INSTALL

Quand ? À la première visite OU quand `sw.js` a changé (même 1 octet)

Ce qui se passe :

1. Le navigateur télécharge `sw.js`
2. L'événement `install` est déclenché
3. C'est le moment idéal pour **pré-cacher** des ressources (optionnel)

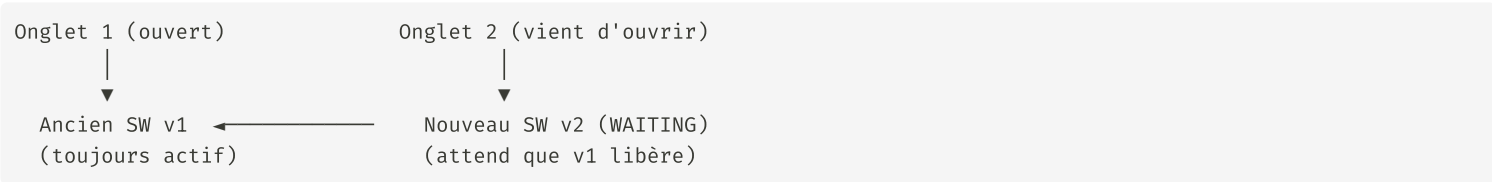
```
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('v1').then(cache => cache.addAll(['/', '/style.css']))
  );
});
```

Notes :

- `event.waitUntil()` → attend la Promise avant de continuer
- Le cache est **optionnel** (le SW peut ne rien cacher)

Cycle de vie : 2. WAITING

Quand ? Après l'installation, SI un ancien SW contrôle encore des onglets



Pourquoi attendre ? Éviter 2 versions simultanées (incohérence des données).

Solutions :

```
self.skipWaiting();  // Force l'activation (⚠️ risque d'incohérence)
```

Bonne pratique : Afficher un message "Nouvelle version disponible, cliquez pour recharger" et laisser l'utilisateur décider.

Démo : tester le SW en "Waiting"

Ouvrir : `demos/demo-sw-waiting/index.html`

Étapes :

1. Ouvrir la page dans **2 onglets**
2. Modifier `sw.js` : changer `VERSION = "2"`
3. Refresh un onglet → DevTools montre "waiting to activate"
4. Le SW v2 est **bloqué** car v1 contrôle encore l'autre onglet !

Tester les solutions :

- **Fermer tous les onglets** → revenir → v2 est actif
- **DevTools** → Application → Service Workers → clic "skipWaiting"
- **Ajouter** `self.skipWaiting()` dans le code

⚠️ Avec `skipWaiting`, les onglets déjà ouverts gardent l'ancienne CSS en mémoire jusqu'au prochain refresh !

Cycle de vie : 2b. Gérer la mise à jour proprement

Le SW peut communiquer avec TOUS les onglets via `clients.matchAll()` :

```
// sw.js — quand le nouveau SW est activé
self.addEventListener('activate', event => {
  self.clients.matchAll().then(clients => {
    clients.forEach(client => client.postMessage({ type: 'SW_UPDATED' }));
  });
});
```

```
// app.js — la page écoute et affiche une bannière
navigator.serviceWorker.addEventListener('message', event => {
  if (event.data.type === 'SW_UPDATED') {
    showBanner('Nouvelle version ! <button onclick="location.reload()">Recharger</button>');
  }
});
```

Résultat : Tous les onglets affichent la bannière, l'utilisateur choisit quand recharger.

Cycle de vie : 3. ACTIVATE

Quand ? Quand le SW sort de WAITING (anciens onglets fermés OU `skipWaiting()`)

Ce qui se passe :

1. L'événement `activate` est déclenché
2. Le SW prend le contrôle des pages
3. Moment idéal pour **nettoyer les anciens caches**

```
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(names => Promise.all(
      names.filter(n => n !== 'v2').map(n => caches.delete(n))
    ))
  );
});
```

Prendre le contrôle de tous les onglets immédiatement :

```
self.addEventListener('activate', event => {
  event.waitUntil(self.clients.claim());
});
```

Cycle de vie : 4. IDLE / Événements

Après activation, le SW est "au repos" (IDLE). Il se réveille sur événement :

Événement	Déclencheur
`fetch`	Requête réseau → répondre depuis le cache
`push`	Notification du serveur → afficher
`sync`	Réseau revient → synchroniser
`message`	La page envoie un message

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(r => r || fetch(event.request))
  );
});
```

Entre les événements → retourne en IDLE (pas de ressources utilisées)

Cycle de vie : 5. TERMINATED

Le navigateur peut "tuer" le SW à tout moment pour économiser les ressources

- Pas d'activité depuis quelques secondes → TERMINATED
- Mémoire insuffisante → TERMINATED
- Navigateur qui optimise → TERMINATED

Conséquences :

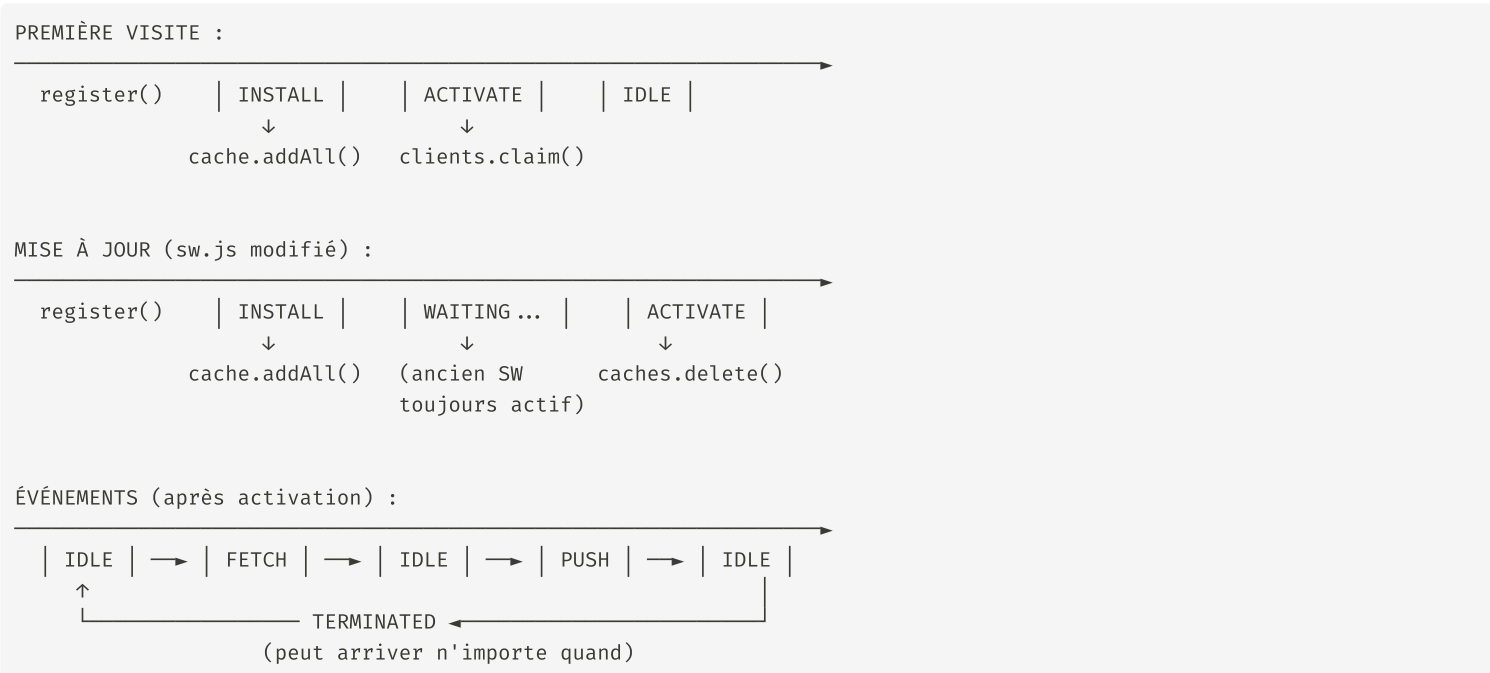
```
// ❌ NE PAS FAIRE : variables globales qui persistent
let compteur = 0;  // Perdu à chaque TERMINATED !

self.addEventListener('fetch', event => {
  compteur++;  // Peut revenir à 0 n'importe quand
});
```

```
// ✅ FAIRE : utiliser IndexedDB ou Cache API pour persister
self.addEventListener('fetch', event => {
  // Lire/écrire dans IndexedDB si besoin de persister
});
```

Le SW est "stateless" → ne jamais compter sur les variables globales

Cycle de vie : résumé visuel



Service Worker : particularités uniques

Ce qui le distingue :

- **Persiste** après fermeture de la page
- **Se réveille** sur événement (push, sync)
- **Intercepte** toutes les requêtes
- **Scope** : contrôle tout un chemin URL
- **HTTPS obligatoire** (sécurité)

Événements qu'il gère :

- `install` – première installation
- `activate` – prise de contrôle
- `fetch` – requête réseau
- `push` – notification push
- `sync` – connexion rétablie
- `message` – communication

Service Worker : enregistrement

```
// main.js

// Vérifier le support
if ('serviceWorker' in navigator) {

  // Enregistrer le Service Worker
  navigator.serviceWorker.register('/sw.js')
    .then(registration => {
      console.log('✅ SW enregistré !');
      console.log('   Scope:', registration.scope);
    })
    .catch(error => {
      console.error('❌ Erreur SW:', error);
    });

} else {
  console.log('Service Worker non supporté');
}
```

Service Worker : sw.js minimal

```
// Événement INSTALL
self.addEventListener('install', event => {
  console.log('🔧 SW: Installation en cours...');
});

// Événement ACTIVATE
self.addEventListener('activate', event => {
  console.log('✅ SW: Activé et prêt !');
});

// Événement FETCH (intercepte les requêtes)
self.addEventListener('fetch', event => {
  console.log('📡 SW intercepte:', event.request.url);

  // Pour l'instant, on laisse passer la requête normalement
  event.respondWith(fetch(event.request));
});
```

Service Worker + Cache : l'aperçu

Le duo qui rend un site offline — l'idée en 12 lignes :

```
const CACHE_NAME = 'ma-pwa-v1';

// À l'installation : remplir le cache
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache =>
      cache.addAll(['/', '/index.html', '/style.css', '/main.js']))
  );
});

// À chaque requête : cache d'abord, réseau sinon
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(r => r || fetch(event.request))
  );
});
```

📅 **Dans la suite du cours (vendredi, Jour 2)** : l'API Cache en détail, les 3 stratégies (Cache First, Network First, SWR), versionning propre. Ce snippet suffit pour les TD du jour.

Service Worker : ce qu’il peut / ne peut pas

✅ **Ce qu’il PEUT faire :**

- Intercepter requêtes réseau
- Gérer le cache (Cache API)
- Recevoir des push notifications
- Exécuter Background Sync
- Utiliser IndexedDB
- Communiquer via postMessage

❌ **Ce qu’il NE PEUT PAS :**

- Accéder au DOM
- Utiliser `localStorage`
- Utiliser `document`, `window`
- Fonctionner sans HTTPS
- Utiliser des APIs synchrones
- Bloquer le thread principal

Support navigateurs & limites vs natif (2026)

Technologie	Chrome/Edge	Firefox	Safari	iOS Safari	Chrome Android
Web Workers	✅	✅	✅	✅	✅
Shared Workers	✅	✅	✅	✅ (16+)	❌
Service Workers / Cache / IndexedDB	✅	✅	✅	⚠️	✅

⚠️ **iOS : ça marche, mais avec des limites** – quota ~50 Mo, éviction après 7 jours d'inactivité, pas de Background Sync, push réservé à l'app **installée**.

Ce qui reste supérieur en natif : push reçu via l'OS (vs navigateur), hardware avancé (certains capteurs), performances 3D/jeux intensifs.

🔒 **Rappel :** Service Worker = HTTPS obligatoire (exception : `localhost`) – détails dans l'annexe Setup. 📅 **Jour 2 :** capacités device (caméra, GPS, vibration…), offline et notifications en pratique.

Côté Android (novembre) : il n'existe pas de Service Worker. Ses trois rôles sont répartis : cache → Room et cache HTTP · arrière-plan → WorkManager · cycle de vie → Activity et process.

Résumé : les 3 Workers

	Web Worker	Shared Worker	Service Worker
Rôle	Calculer sans bloquer	Partager un état entre onglets	Proxy réseau : cache, hors ligne, push
Portée	1 page	Tous les onglets du site	Tout le site, même fermé
Meurt	Avec la page	Avec le dernier onglet	Jamais (arrêté et relancé par le navigateur)

Qui fait quoi ? Cliquer pour vérifier.

- Parser un JSON de 50 Mo sans figer la page ?
- Un panier identique dans tous les onglets ouverts ?
- Le site s'ouvre en mode avion ?
- Une notification arrive, le site est fermé ?
- Une seule connexion WebSocket pour dix onglets ?
- Appliquer un filtre sur une photo de 12 Mpx ?

Annexe : Transferable Objects

Par défaut, `postMessage()` **COPIE les données** – lent pour les gros volumes :

```
const buffer = new ArrayBuffer(100_000_000); // 100 Mo
worker.postMessage(buffer);           // → Copie 100 Mo en mémoire 🐌

// Avec transfert : on DONNE la propriété (instantané, pas de copie)
worker.postMessage(buffer, [buffer]); // → Transfert immédiat ⚡
// ⚠️ buffer est maintenant VIDE côté principal !
```

Utile pour : traitement d'images, audio/vidéo, fichiers binaires volumineux.

Documentation MDN, 3 niveaux de lecture :

Documentation	Contenu
Web Workers API	Guide conceptuel – le "pourquoi"
postMessage()	Référence – le "comment"
Transferable Objects	Optimisation – cas avancés

Chapitre 5

TD : Mise en pratique

Les TD du jour

Dans l'ordre

1. **Mise en place** : serveur local, HTTPS, Node
2. **TD 1** — Web Worker : un calcul qui ne bloque plus la page
3. **TD 2** — Shared Worker : un panier partagé entre onglets
4. **TD 3** — Premier Service Worker : intercepter les requêtes
5. **TD 6** — Mini-PWA « Notes » : page ↔ Service Worker, 100 % hors ligne
6. **TD 7** — Les 3 Workers ensemble

Bonus — pour aller plus loin

- **TD 4** — Cache offline : le site s'affiche sans réseau… et ne se met plus à jour
- **TD 5** — Le Service Worker zombie : vivre le pire bug d'une PWA, puis le réparer

TD indépendants les uns des autres. Frameworks non autorisés : HTML, CSS, JavaScript.

Mise en place technique

Serveur local, HTTPS, déploiement

Pourquoi un serveur web ?

Le problème : ouvrir un fichier HTML directement

```
file:///Users/moi/projet/index.html  ← ❌ NE FONCTIONNE PAS pour les PWA
```

Pourquoi ça ne marche pas ?

- Les Service Workers sont **bloqués** sur le protocole `file://`
- Les requêtes `fetch()` vers des fichiers locaux sont interdites (sécurité)
- Le manifest ne peut pas être chargé correctement
- Pas de simulation réaliste du comportement réseau

La solution : utiliser un serveur HTTP local

```
http://localhost:3000  ← ✅ Fonctionne !
```

HTTP vs HTTPS

Les Service Workers exigent HTTPS (sécurité obligatoire)

```
Un Service Worker peut INTERCEPTER toutes les requêtes
→ Sur HTTP non sécurisé, un attaquant pourrait injecter
   un SW malveillant (man-in-the-middle)
```

Exceptions pour le développement :

URL	Service Worker ?
`https://monsite.com`	✅ Oui
`http://localhost`	✅ Oui (exception dev)
`http://127.0.0.1`	✅ Oui (exception dev)
`http://192.168.1.x`	❌ Non
`http://monsite.com`	❌ Non

→ En local : `localhost` suffit. En production : HTTPS obligatoire.

Tester sur mobile pendant le développement

Le problème :

- Votre serveur tourne sur `http://localhost:3000`
- Votre téléphone ne peut pas accéder à `localhost` de votre PC
- Utiliser l'IP locale (`http://192.168.x.x`) ne permet pas les Service Workers

Les solutions :

Solution	Avantage	Inconvénient
ngrok	Tunnel HTTPS public, fonctionne partout	Compte gratuit requis
Déployer	URL HTTPS permanente	Moins rapide pour itérer
Chrome Remote Debugging	Debug direct sur Android	Config USB requise

Node.js : définition et utilité

Node.js = JavaScript côté serveur

On l'utilise ici pour :

- Lancer un serveur web local (`npx serve`)
- Installer des outils CLI (Firebase, Vercel…)
- Exécuter des scripts de build

npm = gestionnaire de packages Nodenpx = exécuter un package sans l'installer globalement

```
# Avec npm (installe globalement)
npm install -g serve
serve ./mon-projet

# Avec npx (exécute directement, sans install global)
npx serve ./mon-projet
```

→ npx est plus pratique pour des outils qu'on utilise ponctuellement

Installer Node.js

macOS

Option 1 : Installeur officiel

1. Aller sur nodejs.org
2. Télécharger la version **LTS**
3. Lancer l'installeur

Option 2 : Homebrew

```
brew install node
```

Vérifier l'installation :

```
node --version
npm --version
```

Windows

Option 1 : Installeur officiel

1. Aller sur nodejs.org
2. Télécharger la version **LTS**
3. Lancer l'installeur (.msi)
4. ✅ Cocher "Add to PATH"

Option 2 : winget

```
winget install OpenJS.NodeJS.LTS
```

Vérifier (PowerShell) :

```
node --version
npm --version
```

📖 Documentation officielle Node.js | npm Docs

Lancer un serveur local

Méthode la plus simple : `npx serve`

```
# Dans le dossier de votre projet
cd mon-projet-pwa

# Lancer le serveur
npx serve
```

Résultat :

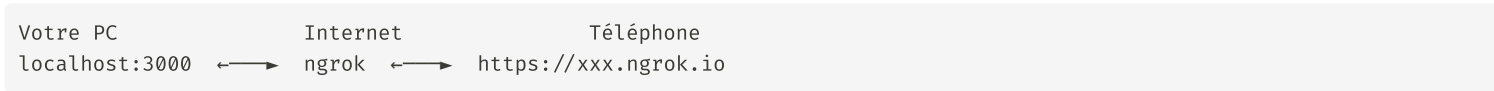
```
   ┌──────────────────────────────────────────┐
   │                                          │
   │  Serving!                                │
   │                                          │
   │  - Local:    http://localhost:3000       │
   │  - Network:  http://192.168.1.42:3000    │
   │                                          │
   └──────────────────────────────────────────┘
```

Autres options :

```
npx serve -p 8080           # Changer le port
npx http-server             # Alternative
python -m http.server 8080  # Si Python installé
```

ngrok : tester sur mobile

ngrok crée un tunnel HTTPS vers votre localhost



Utilisation :

```
# 1. Lancer votre serveur local
npx serve

# 2. Dans un autre terminal, lancer ngrok
ngrok http 3000

# → URL https://xxxx.ngrok.io accessible depuis votre téléphone
```

Prérequis : Compte gratuit sur ngrok.com + `ngrok authtoken VOTRE_TOKEN`

📖 Documentation ngrok

Les options de déploiement gratuit

Service	Méthode	HTTPS	Idéal pour
Netlify	Drag & drop	✅ Auto	Débutants, démo rapide
Vercel	CLI ou Git	✅ Auto	Projets Git, itérations
Firebase Hosting	CLI	✅ Auto	+ Notifications push
GitHub Pages	Git	✅ Auto	Projets open source

Pour ce cours :

- **TD simples** → Netlify (le plus rapide)
- **Projet avec notifications** → Firebase Hosting + FCM

Netlify : déployer en 30 secondes

Méthode drag & drop (sans compte Git)

1. Aller sur app.netlify.com
2. Se connecter (GitHub, GitLab, ou email)
3. Cliquer **"Add new site"** → **"Deploy manually"**
4. **Glisser-déposer** votre dossier de projet
5. ✅ Votre site est en ligne !

```
┌──────────────────────────────────────┐
│  🎉 Votre site est publié !          │
│                                      │
│  https://random-name-123.netlify.app │
└──────────────────────────────────────┘
```

Pour mettre à jour : glisser-déposer à nouveau le dossier

📖 Documentation Netlify

Vercel : déployer avec la CLI

Installation et déploiement

```
# Installer la CLI Vercel
npm install -g vercel

# Dans votre dossier projet
cd mon-projet-pwa

# Déployer (première fois : login + config)
vercel

# Déployer en production
vercel --prod
```

Avantages :

- Déploiement automatique à chaque `git push` (si connecté à GitHub)
- Preview URLs pour chaque branche/PR
- Rollback facile

📖 Documentation Vercel CLI

Firebase Hosting : pour les notifications push

Pourquoi Firebase ?

- Hébergement gratuit avec HTTPS
- **Firebase Cloud Messaging (FCM)** pour les notifications push
- Tout-en-un : hosting + backend + notifications

Installation et déploiement

1. Installer Firebase CLI

```
npm install -g firebase-tools
```

2. Se connecter

```
firebase login
```

3. Initialiser le projet

```
firebase init hosting
```

4. Configurer

- Choisir/créer un projet Firebase
- Dossier public : `./` ou `./dist`
- Single-page app : selon votre projet

5. Déployer

```
firebase deploy
```

📖 Documentation Firebase Hosting

Firebase : créer un projet

Étapes dans la console Firebase :

1. Aller sur console.firebase.google.com
2. Cliquer **"Créer un projet"**
3. Nommer le projet (ex: `pwa-cours-m2`)
4. Désactiver Google Analytics (optionnel pour le cours)
5. Attendre la création (~30 secondes)

Ensuite, activer Hosting :

1. Dans le menu gauche : **Build → Hosting**
2. Cliquer **"Get started"**
3. Suivre les instructions (install CLI, init, deploy)

Récapitulatif : workflow de développement

```
1. DÉVELOPPER
   └→ Éditer le code dans votre éditeur (VS Code ... )

2. TESTER EN LOCAL
   └→ npx serve → http://localhost:3000

3. TESTER SUR MOBILE (optionnel)
   └→ ngrok http 3000 → https://xxx.ngrok.io

4. DÉPLOYER
   └→ Netlify (drag & drop) ou Firebase (firebase deploy)

5. PARTAGER
   └→ URL HTTPS publique → tester sur n'importe quel device
```

Checklist avant de commencer le TD

✅ Vérifications :

- ☐ Node.js installé (`node --version`)
- ☐ npm fonctionne (`npm --version`)
- ☐ Éditeur de code prêt (VS Code recommandé)
- ☐ Chrome installé (pour DevTools)
- ☐ Compte Netlify ou Firebase créé

📁 Structure projet minimale :

```
mon-projet-pwa/
├── index.html
├── style.css
├── app.js
├── sw.js          ← Service Worker
└── manifest.json  ← Web App Manifest
```

Commandes utiles :

```
npx serve                    # Serveur local
firebase deploy              # Déployer sur Firebase
vercel                       # Déployer sur Vercel
```

Un mot sur l'IA

ChatGPT **Claude** **Gemini** **Copilot** **Grok** **Mistral**



Récapitulatif : les Workers

À quoi sert chacun des trois ?

- **Web Worker** : calculer sans bloquer l'interface
- **Shared Worker** : partager un état entre onglets
- **Service Worker** : intercepter le réseau, servir depuis le cache, recevoir un push

Quels pièges ?

- Un SW qui sert **l'ancienne version** après déploiement (le « zombie » du TD 5)
- Un cache qu'on **ne sait plus invalider**
- `skipWaiting()` qui active un SW au milieu d'une session
- Le **scope** : un SW à `/app/sw.js` ne contrôle pas `/`
- HTTPS obligatoire, `localhost` excepté
- Pas de DOM, pas de `localStorage` dans un worker

Quand ne pas en mettre ?

- Un calcul de quelques millisecondes : le fil principal suffit
- Un appel réseau : `fetch` + `await` **attendent** déjà sans bloquer
- Un site vitrine sans besoin hors ligne : un SW, c'est un cache de plus à maintenir

Pour la discussion : un utilisateur signale « le site ne se met pas à jour, même après F5 ». Quelle couche est en cause, et comment le prouver avec l'onglet Network ?

TD 1 : Web Worker

Objectif : un calcul lourd qui ne bloque plus la page — et le voir.

Exercice :

1. Créer `index.html` avec un champ N, deux boutons et **deux témoins** : un carré en rotation (animation CSS) et un bouton « Compteur » qui s'incrémente au clic
2. Bouton 1 : calculer la somme des √i de 1 à N **dans la page** → le carré s'arrête, le compteur ne répond plus
3. Créer `worker.js` qui fait le même calcul ; bouton 2 : l'exécuter **dans un Worker** (`postMessage` / `onmessage`) → le carré tourne, le compteur répond
4. Afficher le résultat et le temps de calcul dans les deux cas

Validation : même durée dans les deux cas (≈ 2 à 4 s pour N = 1 000 000 000) ; seule la page change de comportement.

TD 2 : Shared Worker - Panier partagé

Objectif : Synchroniser un panier e-commerce entre plusieurs onglets

Scénario : L'utilisateur ouvre 2 onglets de votre boutique. Il ajoute un produit dans l'onglet 1 → l'onglet 2 doit se mettre à jour instantanément !

> ⚠️ **Chrome Android ne supporte PAS les Shared Workers !** Testez sur Firefox, Safari, ou desktop.

Fichiers à créer : `index.html`, `app.js`, `shared-worker.js`

TD 2 : Shared Worker - Aide

```
// shared-worker.js
let panier = [];
let ports = [];

onconnect = (e) => {
  const port = e.ports[0];
  ports.push(port);

  port.onmessage = (event) => {
    if (event.data.action === "add") {
      panier.push(event.data.produit);
      ports.forEach((p) => p.postMessage({ panier })); // Broadcast
    }
  };
  port.postMessage({ panier }); // État initial
};
```

Test : Ouvrir 2 onglets (Firefox/Safari/Desktop), ajouter un produit → les 2 se mettent à jour !

🚀 **Bonus :** Formulaire multi-onglets (draft partagé) – commencer à écrire dans un onglet, continuer dans l'autre !

TD 3 : Premier Service Worker

Objectif : Enregistrer un SW et observer son comportement

Exercice :

1. Créer un SW minimal (install, activate, fetch)
2. Logger toutes les requêtes interceptées
3. Observer dans DevTools → Application → Service Workers

Lancer un serveur local :

```
python -m http.server 8000
# ou
npx serve
```

💡 **Rappel :** les logs du SW ne sont pas dans la console de la page → Console → menu "top" → **sw.js**, ou Application → Service Workers → "Inspect".

TD 3 : Bonus - Fallback si fetch échoue

Retourner une réponse par défaut si `/api/users` échoue :

```
self.addEventListener("fetch", (e) ⇒ {
  if (e.request.url.includes("/api/users")) {
    e.respondWith(
      fetch(e.request).catch(
        () ⇒
          new Response(JSON.stringify({ user: "unknown" }), {
            headers: { "Content-Type": "application/json" },
          }),
      ),
    );
  }
});
```

TD 6 : Mini-PWA "Notes"

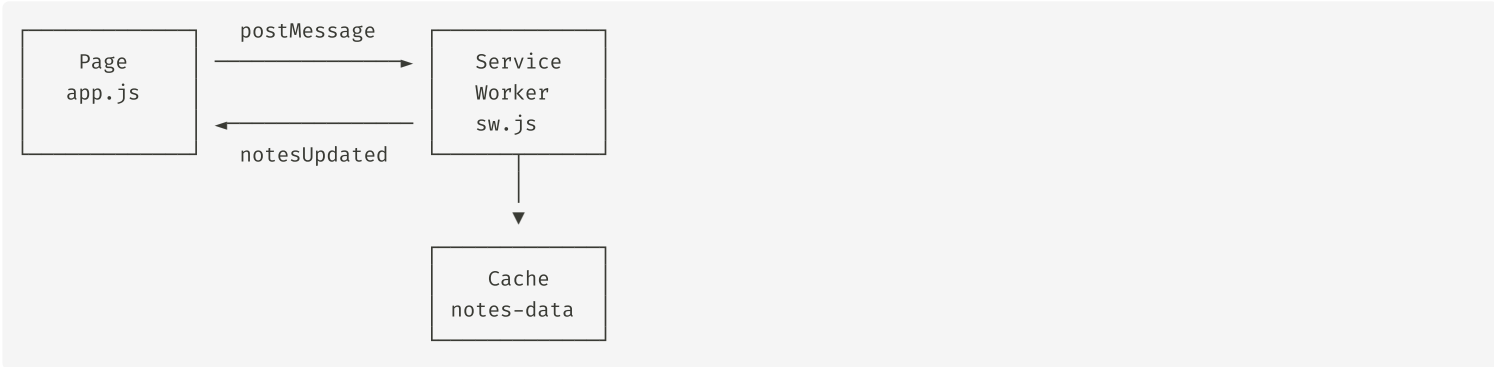
Objectif : Créer une PWA avec communication Page ↔ Service Worker

Fonctionnalités :

- Ajouter des notes via formulaire
- Notes stockées dans le **Cache** (pas localStorage !)
- Communication via `postMessage`
- Fonctionne 100% offline

🔮 **Note :** stocker des données via l'API Cache est un bricolage assumé — l'outil adapté n'a pas encore été vu. Vendredi (Jour 2) : **IndexedDB**, la vraie base de données du navigateur — cette app y est refaite proprement (TD PWA Notes).

TD 6 : Architecture



Pourquoi le Cache plutôt que localStorage ?

- Plus de stockage (50MB+ vs 5MB)
- Accessible depuis le SW
- Peut stocker images, blobs…

TD 6 : Aide - app.js

```
// Envoyer une note au SW
form.addEventListener("submit", (e) ⇒ {
  e.preventDefault();
  navigator.serviceWorker.controller.postMessage({
    action: "addNote",
    note: input.value,
  });
});

// Écouter les réponses du SW
navigator.serviceWorker.addEventListener("message", (e) ⇒ {
  if (e.data.action ═══ "notesUpdated") {
    render(e.data.notes);
  }
});
```

TD 6 : Aide - sw.js

```
self.addEventListener("message", async (e) => {
  if (e.data.action === "addNote") {
    const cache = await caches.open("notes-data");
    // ... récupérer notes existantes, ajouter la nouvelle ...

    // Notifier TOUS les onglets
    const clients = await self.clients.matchAll();
    clients.forEach((client) => {
      client.postMessage({ action: "notesUpdated", notes });
    });
  }
});
```

Bonus : Les notes se synchronisent entre onglets (comme le Shared Worker !)

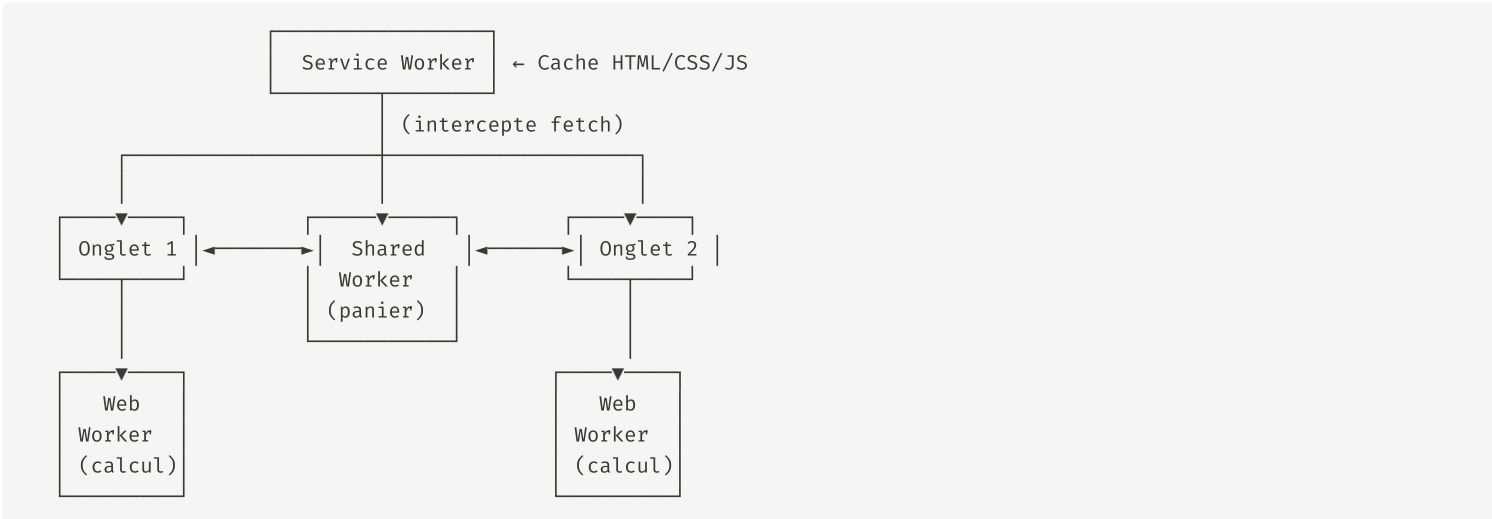
TD 7 : Les 3 Workers ensemble

Objectif : Créer une mini-boutique utilisant les 3 types de workers

🔧 **Web Worker**	🔗 **Shared Worker**	📦 **Service Worker**
Calcul de réduction personnalisée (algo lourd sans bloquer l'UI)	Panier partagé entre tous les onglets	Cache offline + page fallback

Test final : Ouvrir 2 onglets, ajouter au panier, calculer promo, passer en offline → tout fonctionne !

TD 7 : Architecture



Bonus

TD 4 et TD 5 : le cache et son piège

TD 4 : Cache offline

Objectif : Faire fonctionner un site sans connexion

Exercice :

1. À l'install, mettre en cache : HTML, CSS, JS
2. Au fetch, répondre depuis le cache si disponible
3. Tester : DevTools → Network → Offline

Test de validation :

La page doit s'afficher même en mode Offline !

TD 4 : Le piège du cache

Tester le problème :

1. Charger la page → CSS en cache
2. Modifier `style.css` : `h1 { color: red; }`
3. Refresh → **toujours l'ancienne couleur !**

Pourquoi ? Le SW sert depuis le cache, il ignore que le fichier a changé.

Solution manuelle : DevTools → Application → Cache Storage → clic droit → Delete

Vraie solution → TD 5 (versionnage + nettoyage à l'activation)

TD 5 : Le Service Worker zombie

Objectif : vivre le pire bug d'une PWA — un site impossible à mettre à jour — puis le réparer.

Phase 1 — le zombie *(décocher « Update on reload » dans DevTools → Application → Service Workers)*

Point de départ : `TD/td5-zombie` — cache-first sur tout, mise en cache au fil de l'eau, `CACHE_NAME = "v1"`, **aucun nettoyage**.

1. Charger la page, recharger une fois : tout est en cache
2. Modifier `style.css` (couleur du h1) → recharger → **rien ne change** *(le `fetch` sert le cache)*
3. Modifier `sw.js` (ajouter un `console.log`) → recharger → le nouveau SW s'installe, reste en **waiting** ; toujours rien
4. Fermer **tous** les onglets, rouvrir → le nouveau SW est actif… et le site n'a **toujours pas changé**

Pourquoi ? Le cache `v1` n'est jamais invalidé : chaque fichier déjà vu est servi à vie, et le nouveau SW ne remplace rien.
L'utilisateur est bloqué définitivement — sauf s'il vide son cache à la main, ce qu'aucun utilisateur ne fait.

Se libérer : DevTools → Application → Service Workers → *Unregister* + Cache Storage → *Delete*

TD 5 : Phase 2 — la délivrance

Refaire proprement *(corrigé : `TD/td5-versionning`)* :

1. **Pré-cacher à l'** `install` avec `cache.addAll()` et **versionner** : `CACHE_NAME = "v2"`, incrémenté à **chaque** déploiement
2. À l' `activate`, **supprimer les anciens caches** :

```
self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(
        keys.filter((key) => key !== CACHE_NAME).map((key) => caches.delete(key)),
      ),
    ),
  );
});
```

3. Tester : modifier le CSS + passer en `v3` → recharger, fermer/rouvrir → la modification arrive
4. **Bonus :** `skipWaiting()` + `clients.claim()` pour activer sans fermer les onglets (cf. cours)
5. **Ne jamais mettre** `sw.js` **dans la liste des fichiers cachés**

La gestion des mises à jour n'est pas un détail : c'est la différence entre une PWA professionnelle et un piège à utilisateurs. La suite au Jour 3 : la bannière « nouvelle version ».

Récap Jour 1

Histoire : iPhone 2007 → App Store 2008 → PWA 2015

PWA : Concept Google, technologies W3C, 3 piliers

Workers :

- Web Worker → calculs parallèles
- Shared Worker → sync onglets
- Service Worker → offline, cache, push

Vendredi (Jour 2) : API Cache, IndexedDB, stratégies, Performance

Questions ?

À vendredi !

Vendredi 9 octobre – Performance, Manifest, Cache, IndexedDB, Offline, Notifications